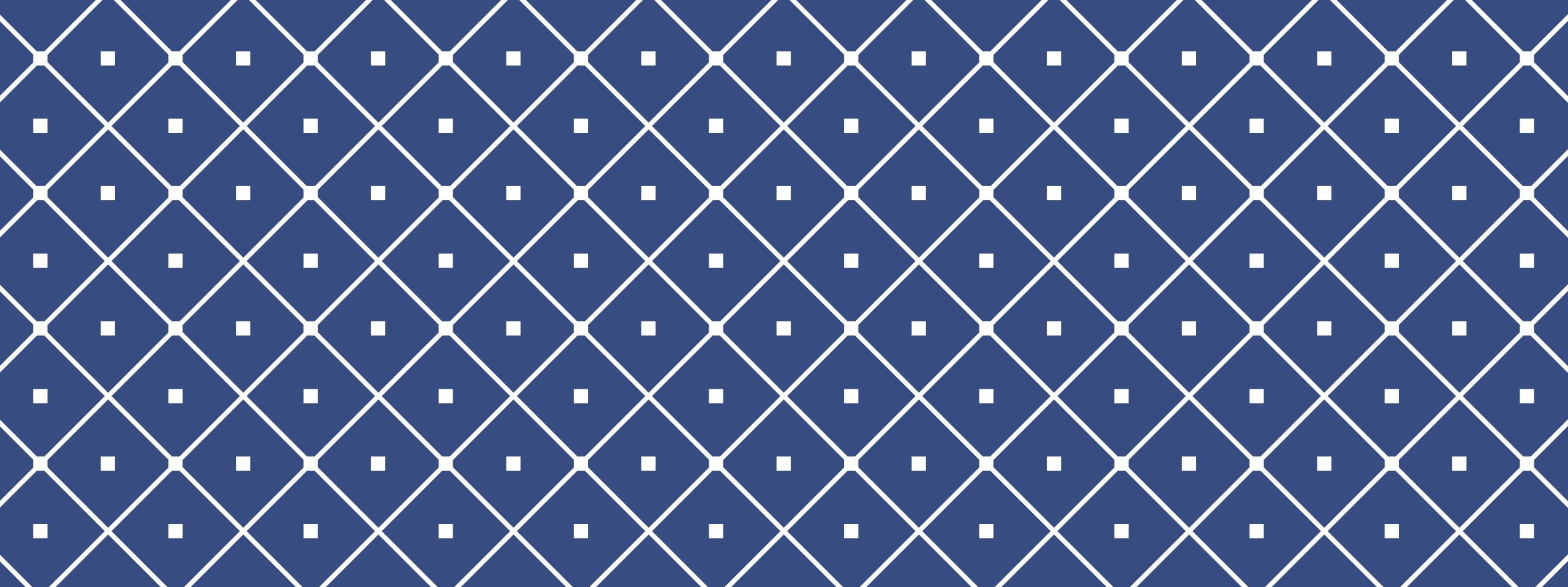




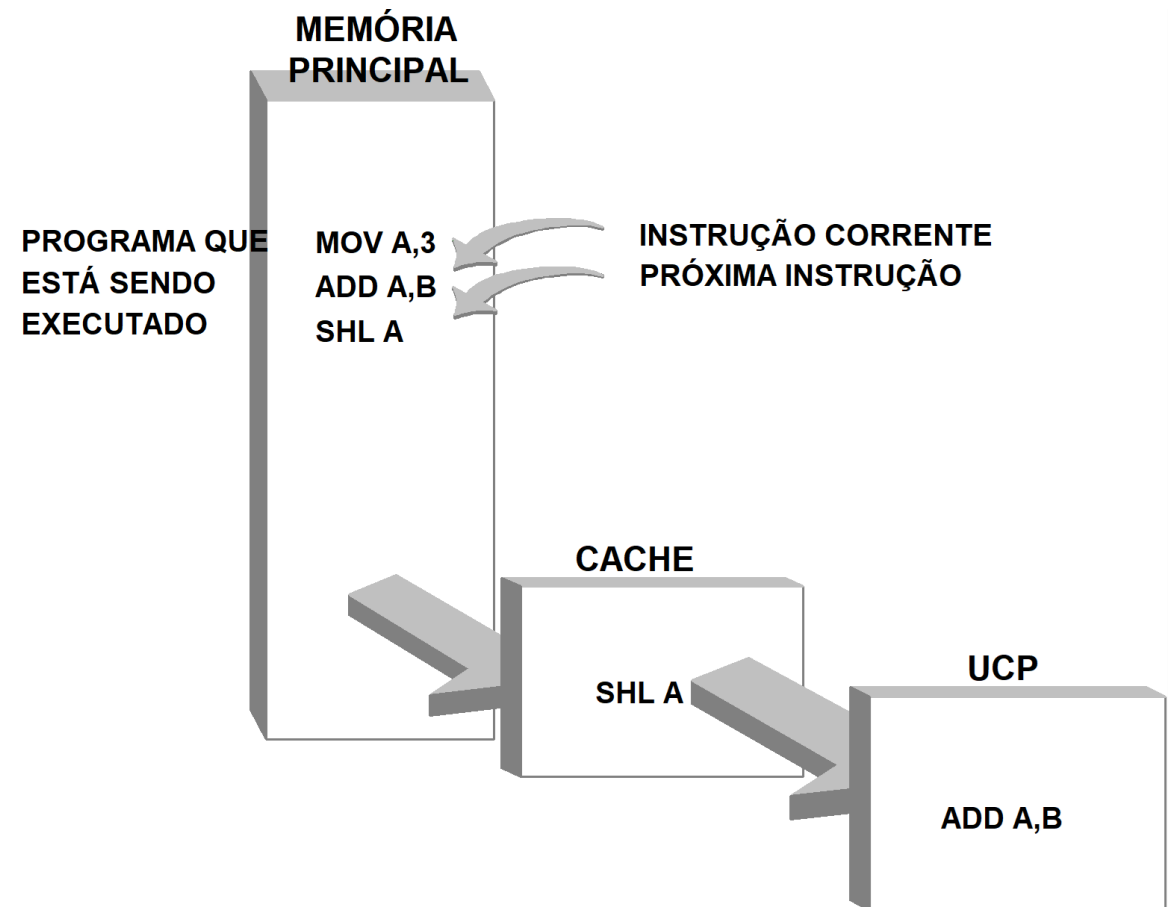
MEMÓRIAS CACHE

Prof. André Gustavo Adami

INTRODUÇÃO

Memória colocada entre a Unidade Central de Processamento e a memória principal

- Reduzir a diferença de tempo entre o processamento da UCP e a operação da memória principal
- Traz os dados/instruções para mais próximo da UCP (Hierarquia de Memória)



INTRODUÇÃO

Utiliza-se memória SRAM na implementação de memórias cache

- Mais rápida que DRAM (memória principal)

Quantidade de memória cache é sempre menor do que a memória principal para não inviabilizar o seu uso

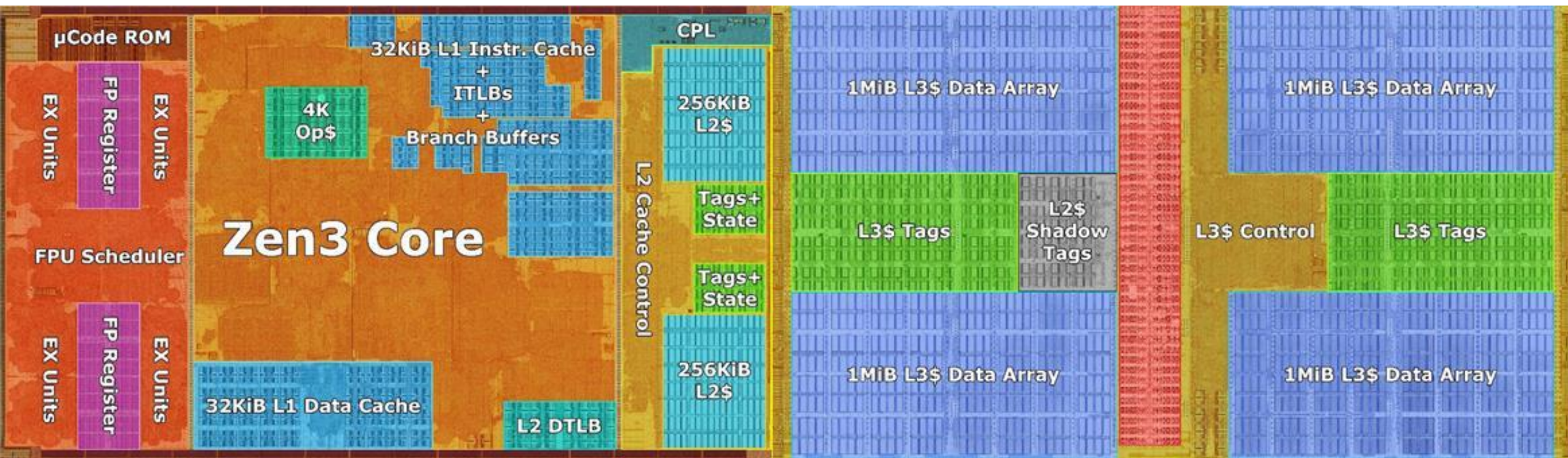
 Table of Common System Latencies	Time (nanoseconds)	CPU Cycles (@ 4 GHz)	Time (At 1 Hz)
TCP packet retransmit delay	3,000,000,000	12,000,000,000	381 years
Internet: SF to NY	40,000,000	160,000,000	5.1 years
5400 RPM HDD access	17,430,000	69,720,000	2.2 years
7200 RPM HDD access	11,920,000	47,680,000	552 days
NAND Flash SSD access	80,000	320,000	88.9 hours
3D Xpoint SSD access	10,000	40,000	11.1 hours
DDR4-3600 SDRAM access	68.1	272.4	4.5 minutes
CPU L3 Cache access	12.8	51.2	51.2 seconds
CPU L2 Cache access	2.8	11.2	11.2 seconds
CPU L1 Cache access	0.9	3.6	3.6 seconds
one CPU Cycle (4 GHz)	0.25	1	1 second

<https://hothardware.com/news/cpu-cache-explained>

INTRODUÇÃO

Utiliza-se memória SRAM na implementação de memórias cache

- Mais rápida que DRAM (memória principal)



PRINCÍPIOS

Localidade Temporal

- Ao referenciar uma posição de memória, é provável que haja referências futuras a mesma posição referenciada (por exemplo, variável de controle de um laço)
- Mantenha itens de dados mais recentemente acessados nos níveis da hierarquia mais próximos do processador

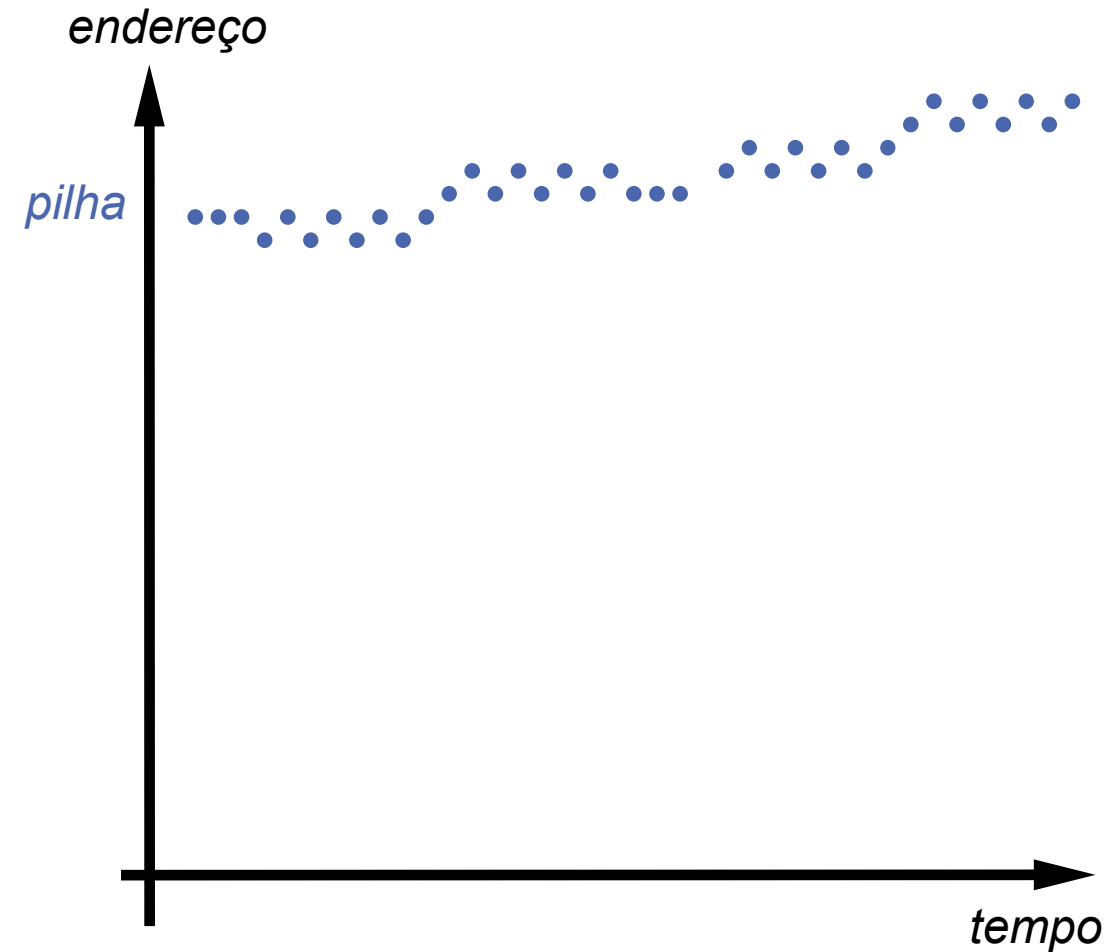
Localidade Espacial

- Ao referenciar uma posição de memória, é provável que haja referências futuras aos vizinhos da posição referenciada (por exemplo, vetor/matriz)
- Mova blocos de palavras contíguas para os níveis da hierarquia mais próximos do processador

PRINCÍPIOS

Acesso à memória apresenta determinados padrões conforme o elemento a ser acessado

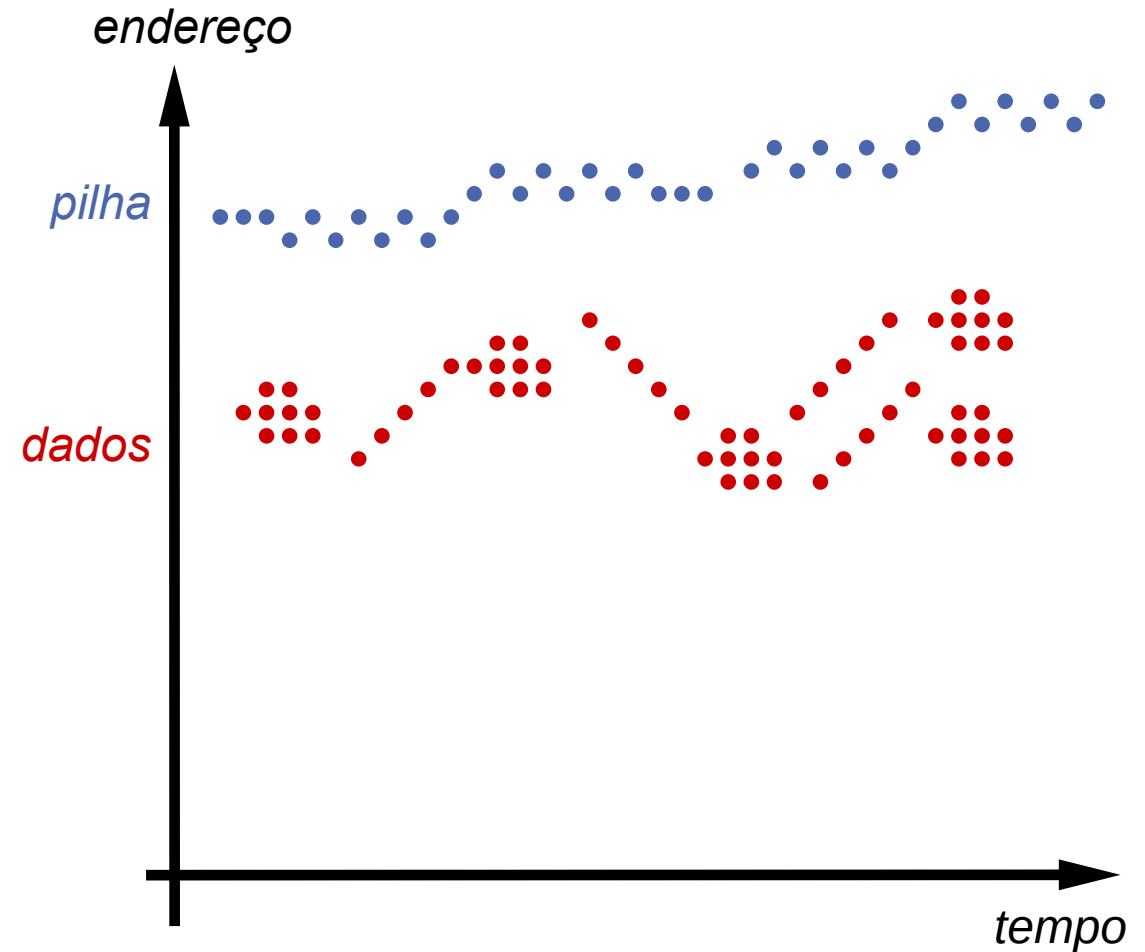
- **Pilha** é utilizada para armazenamento temporário de dados (salvar dados de registros, variáveis locais, endereços de retorno, entre outros) em programação de baixo nível. Endereçamento é sempre sequencial, pois as únicas operações são empilhar e desempilhar



PRINCÍPIOS

Acesso à memória apresenta determinados padrões conforme o elemento a ser acessado

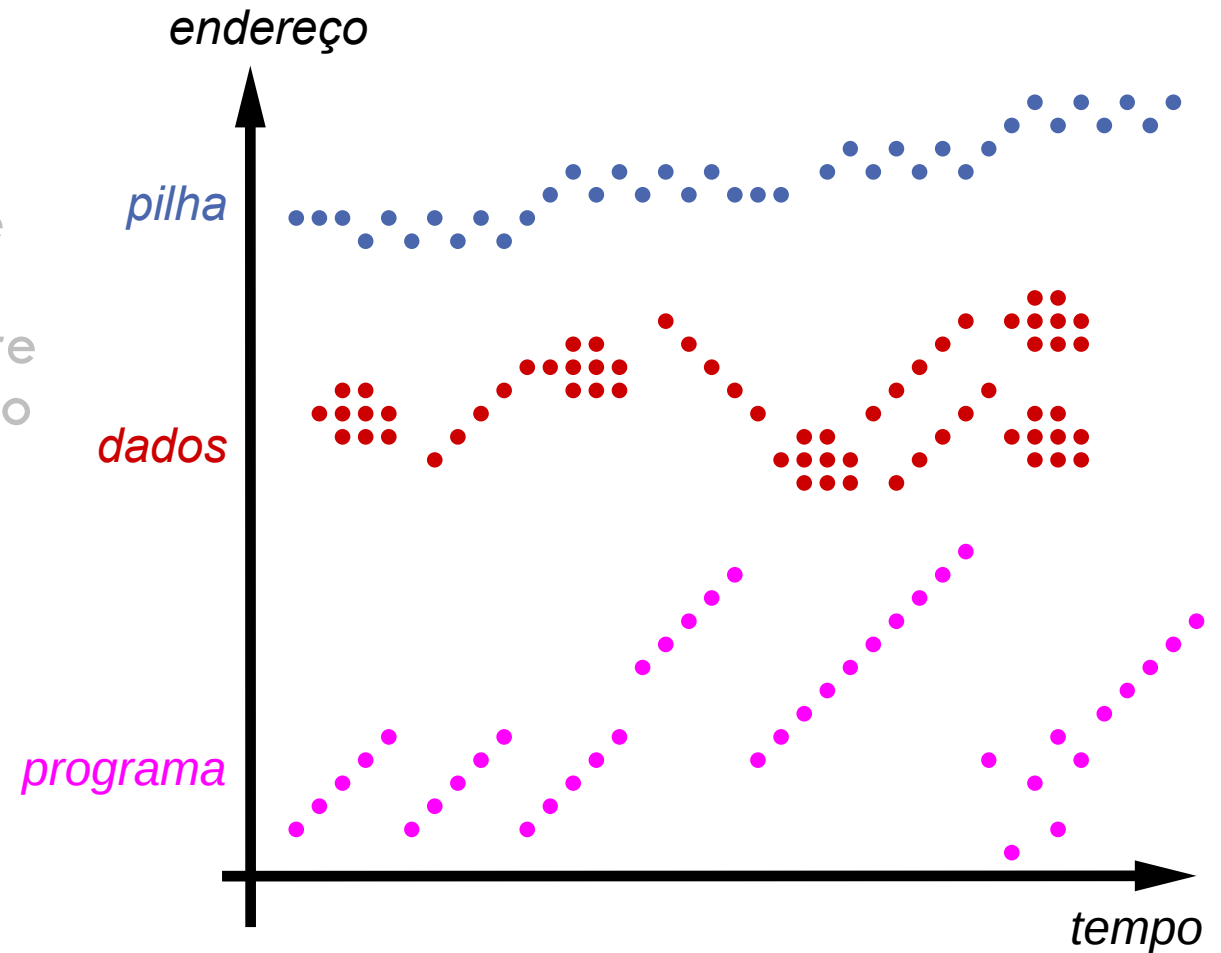
- Pilha é utilizada para armazenamento temporário de dados (salvar dados de registros, variáveis locais, endereços de retorno, entre outros) em programação de baixo nível. Endereçamento é sempre sequencial, pois as únicas operações são empilhar e desempilhar
- **Dados** são acessados de forma que o mesmo endereço (e seus vizinhos...) é utilizado várias vezes repetidas vezes



PRINCÍPIOS

Acesso à memória apresenta determinados padrões conforme o elemento a ser acessado

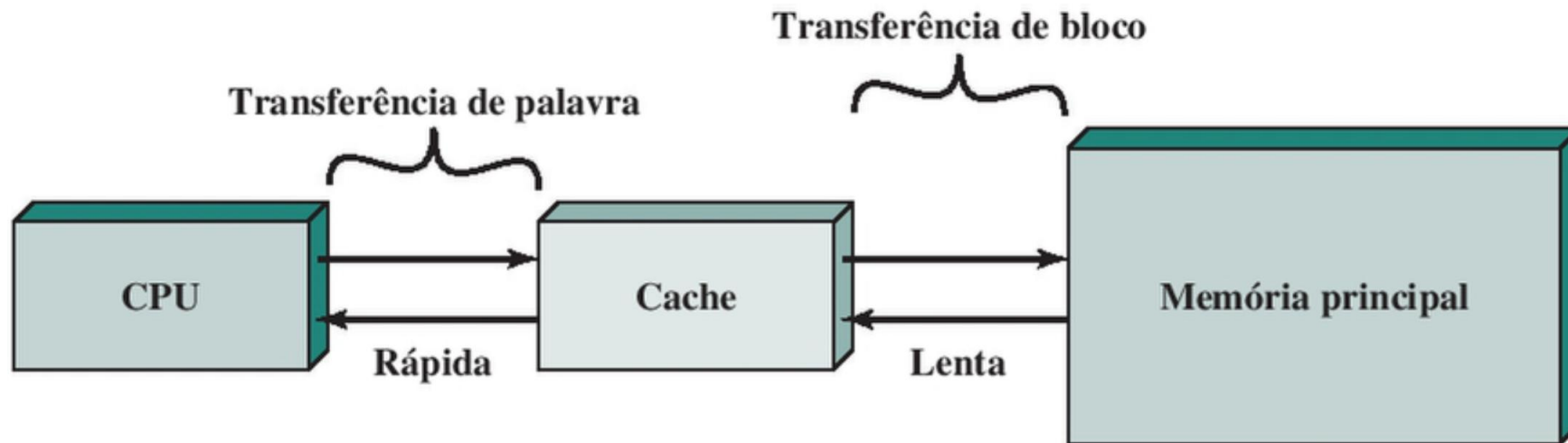
- Pilha é utilizada para armazenamento temporário de dados (salvar dados de registros, variáveis locais, endereços de retorno, entre outros) em programação de baixo nível. Endereçamento é sempre sequencial, pois as únicas operações são empilhar e desempilhar
- Dados são acessados de forma que o mesmo endereço (e seus vizinhos...) é utilizado várias vezes repetidas vezes
- **Programas** apresentam um acesso geralmente sequencial (máquina von Neumann)



ESTRUTURA

Para garantir o princípio da localidade espacial, blocos de bytes (instruções ou dados) são transferidos entre a memória principal e a memória cache

A **memória cache** é formada por um **conjunto de blocos de bytes**



ESTRUTURA

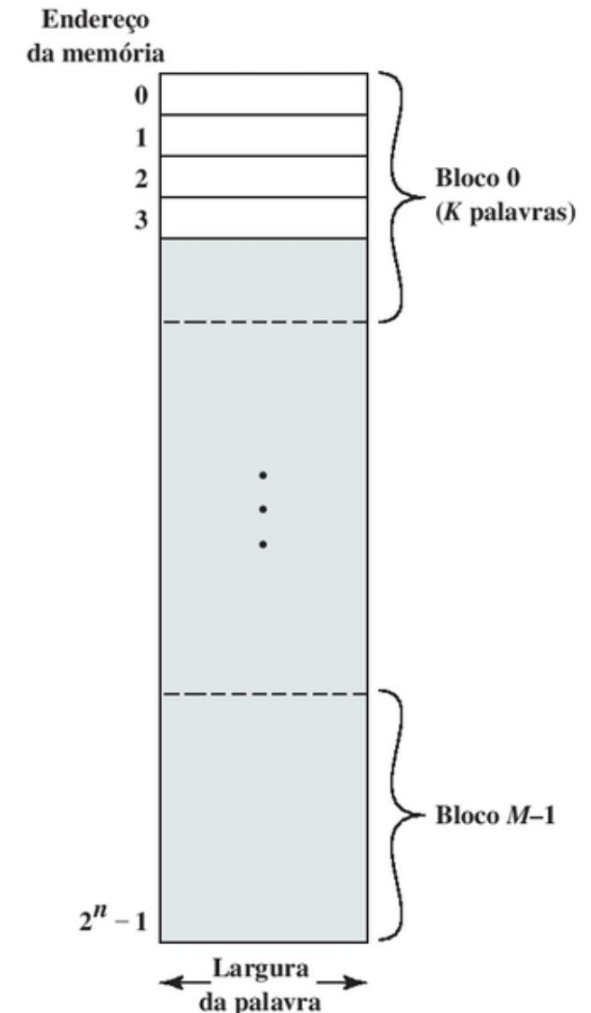
A Memória Principal (MP) consiste em até 2^n palavras endereçáveis, com um endereço de n bits para cada palavra

A MP é considerada um conjunto de blocos de tamanho fixo com K palavras cada

- Existem $M = 2^n / K$ blocos na memória principal

Por exemplo, uma memória principal com 8 GB

- Assumindo um bloco de 64 bytes, a memória principal possui 128M blocos ($8\text{GB} / 64\text{B} = 2^{33} / 2^6 = 2^{27}$) de 64 bytes



ESTRUTURA

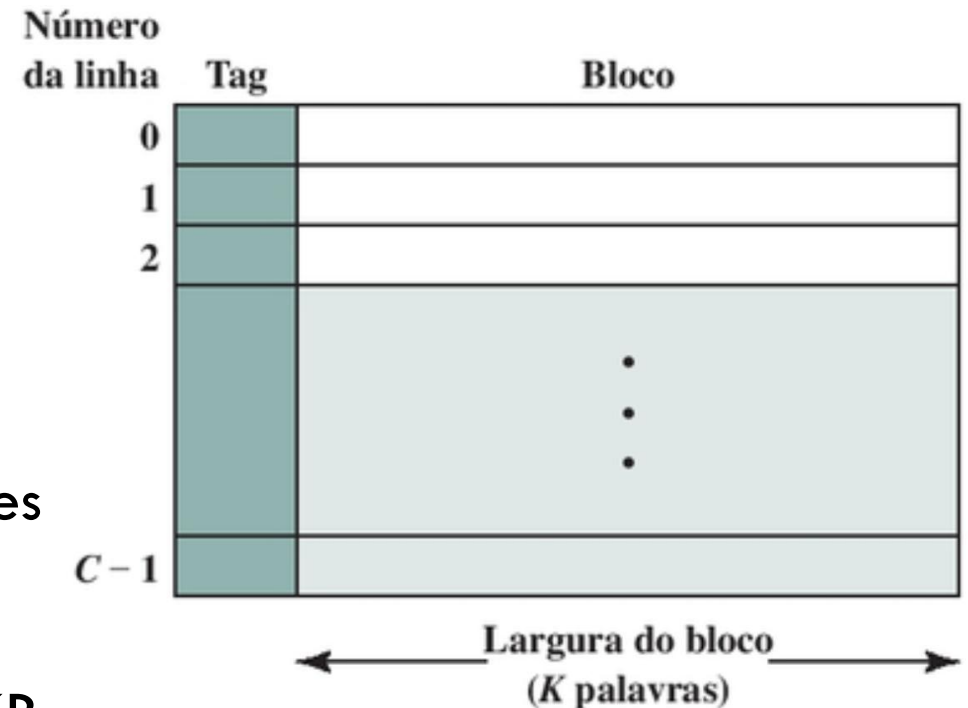
A **memória cache** consiste em **C** blocos, chamados de **linhas**

- $C \ll M$

Cada **linha** é formada por um **bloco de bytes** de tamanho fixo (**K** palavras) da memória e outros bits de controle e identificação

- Tamanho da linha == tamanho do bloco de bytes

Por exemplo, uma memória cache de 64 KB com um bloco de 64 bytes possui 1024 ($64K/64$) blocos de 64 bytes

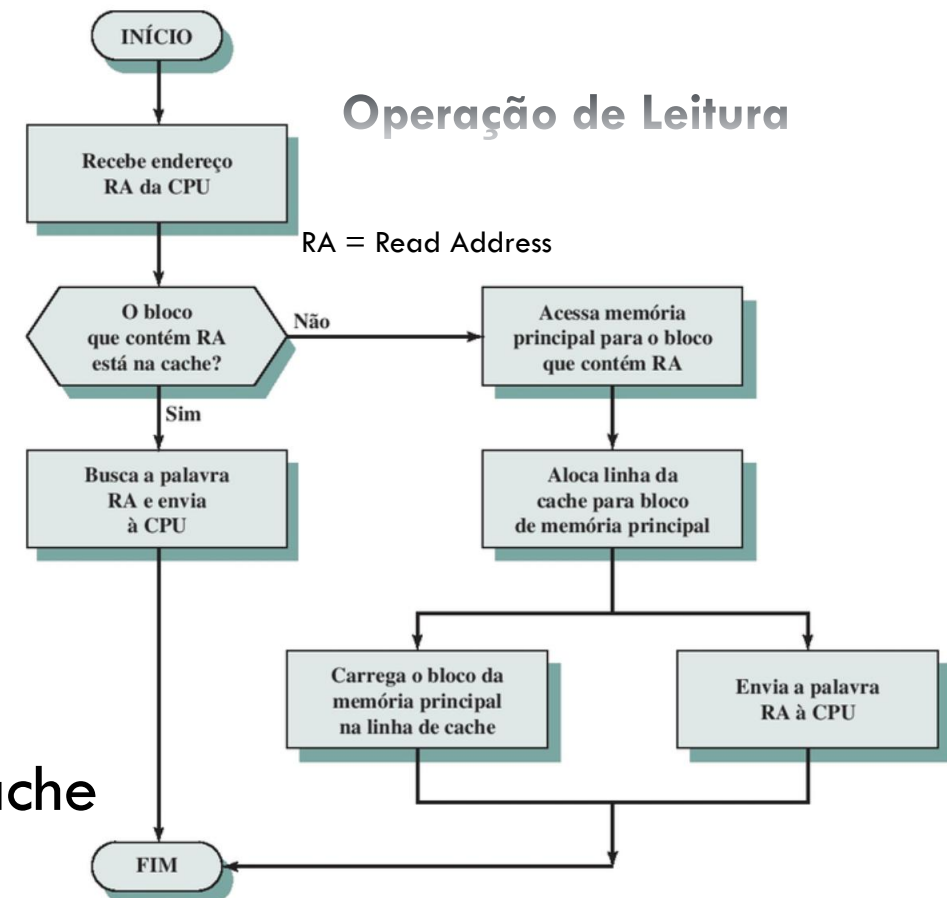


OPERAÇÃO

O acesso à cache é realizado a cada operação de leitura ou escrita

1. CPU requisita conteúdo do local de memória
2. Verifica se os dados estão em cache
3. Se estiverem, obtém o valor da cache (rápido)
4. Se não, lê bloco solicitado da memória principal para a cache
5. Depois, entrega da cache à CPU

Cache inclui *tags* para identificar qual bloco da memória principal está em cada linha da cache

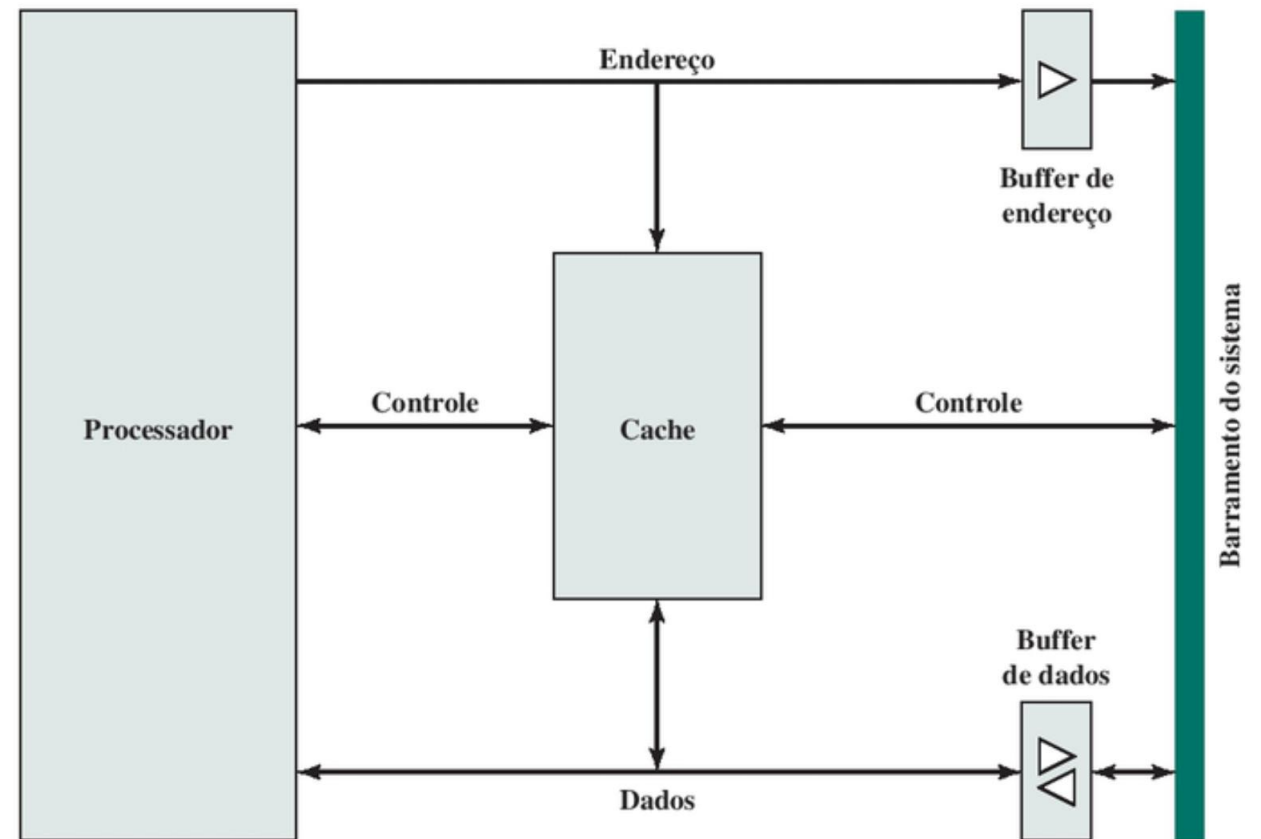


OPERAÇÃO

Um **acerto de cache** (*cache hit*) ocorre quando a busca do endereço é na cache

- Os buffers de endereço e de dados são desativados e a comunicação é apenas entre o processador e a memória cache

Caso contrário, ocorre um **erro de cache** (*cache miss*)



DESEMPENHO

Em uma memória de dois níveis, M1 é uma memória de nível superior (mais próxima da UCP), menor, mais rápida e mais cara do que a memória de nível inferior (M2)

Tempo médio de acesso (sistema)

$$\begin{aligned} t_s &= h \times t_1 + (1 - h) \times (t_1 + t_2) \\ &= t_1 + (1 - h) \times t_2 \end{aligned}$$

onde

h = taxa de acerto

t_1 = tempo de acesso da M1 (cache, cache de disco,..)

t_2 = tempo de acesso de M2 (memória principal, disco)

DESEMPENHO

Por exemplo, suponha uma memória cache com tempo de acesso de **20 ns** e uma memória principal com tempo de acesso de **80 ns** e uma taxa de acerto de 80% (dados típicos de um computador PC 80486 com 256 KB de cache)

$$\begin{aligned}t_s &= t_1 + (1 - h) \times t_2 \\&= 20 + (1 - 0,8) \times 80 \\&= 20 + 0,2 \times 80 \\&= 36 \text{ ns} < 80 \text{ ns}\end{aligned}$$

Nesse caso, o tempo médio de acesso aos dados cai para menos da metade com o uso da cache

PROJETO

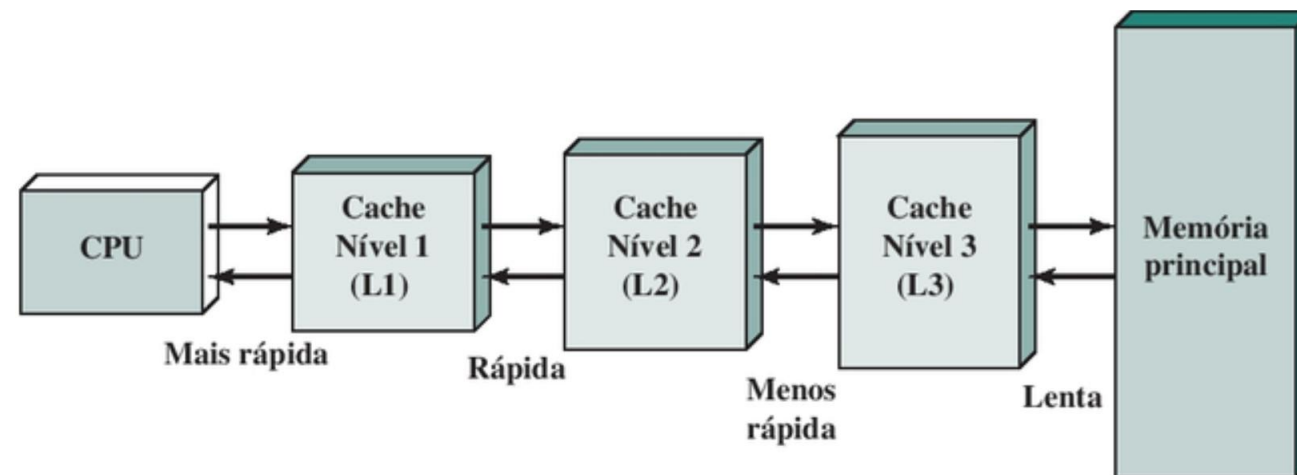
1. Número de Memórias Cache
2. Capacidade da Cache
3. Tamanho do Bloco de Dados
4. Endereço de Cache
5. Função de Mapeamento
6. Algoritmo de Substituição
7. Política de Escrita
8. Política de Coerência...

NÚMERO DE CACHES

Quando lançadas, os sistemas de memória possuíam somente uma cache

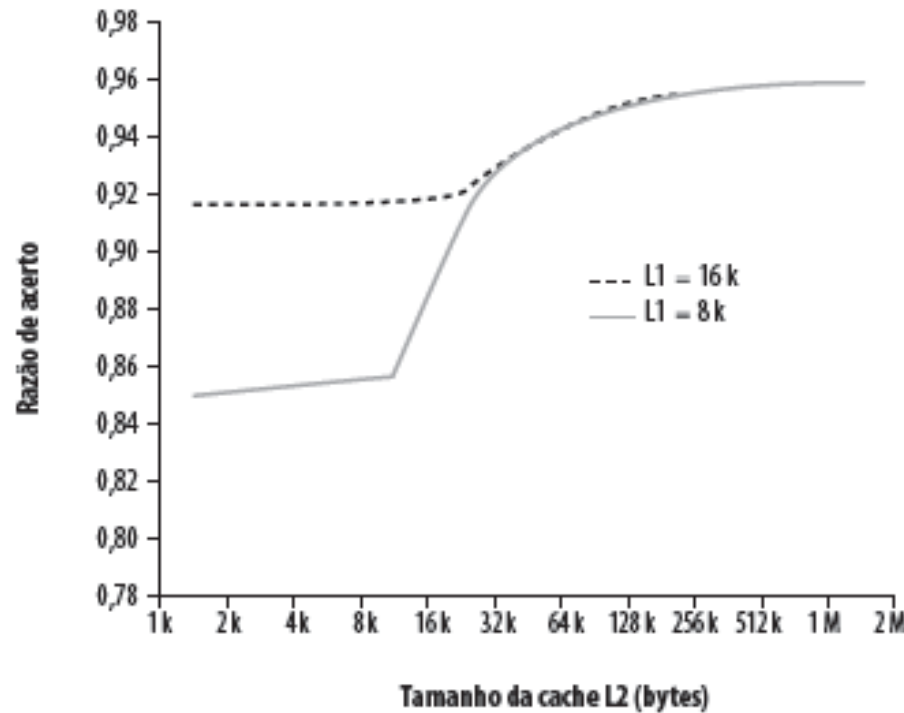
Alta densidade lógica permite caches no chip (multinível caches)

- Mais rápido que acesso ao barramento
- Libera barramento para outras transferências (cache externa)



NÚMERO DE CACHES

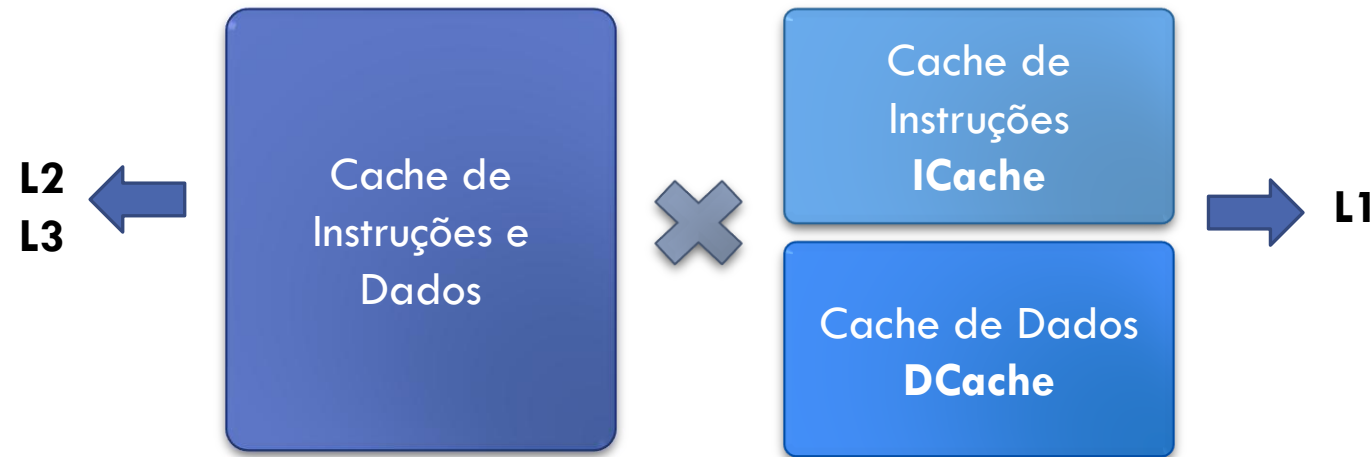
A cache L2 deve ser maior que a cache L1 para ter efeito no desempenho



Razão de acerto total (L1 & L2) para uma cache L1 de 8 KB e 16 KB

NÚMERO DE CACHES

No princípio, as instruções e dados eram armazenados em uma única cache (von Neumann)



Ao longo do tempo, os projetos separavam as instruções e dados em duas caches (Harvard) que se interligavam com o processador

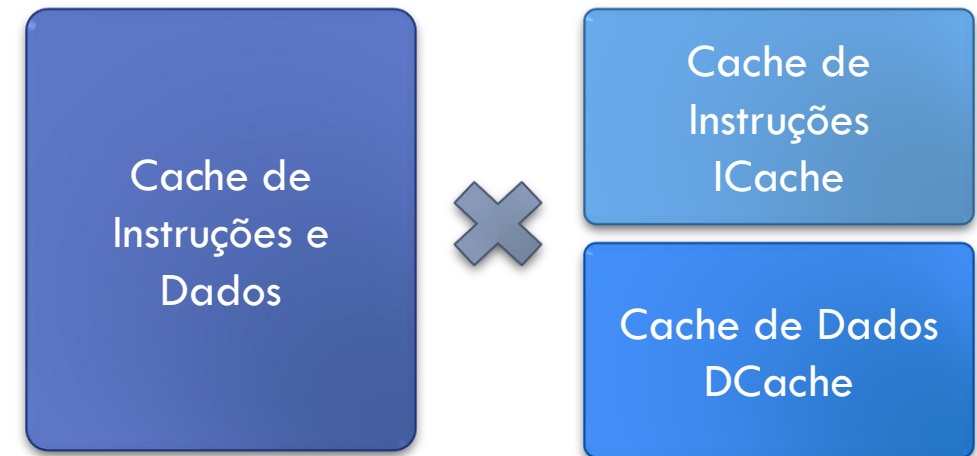
NÚMERO DE CACHES

Vantagens da cache unificada

- Maior taxa de acerto
 - Equilibra carga entre buscas de instrução e dados
- Apenas uma cache para projetar e implementar

Vantagens da cache separada

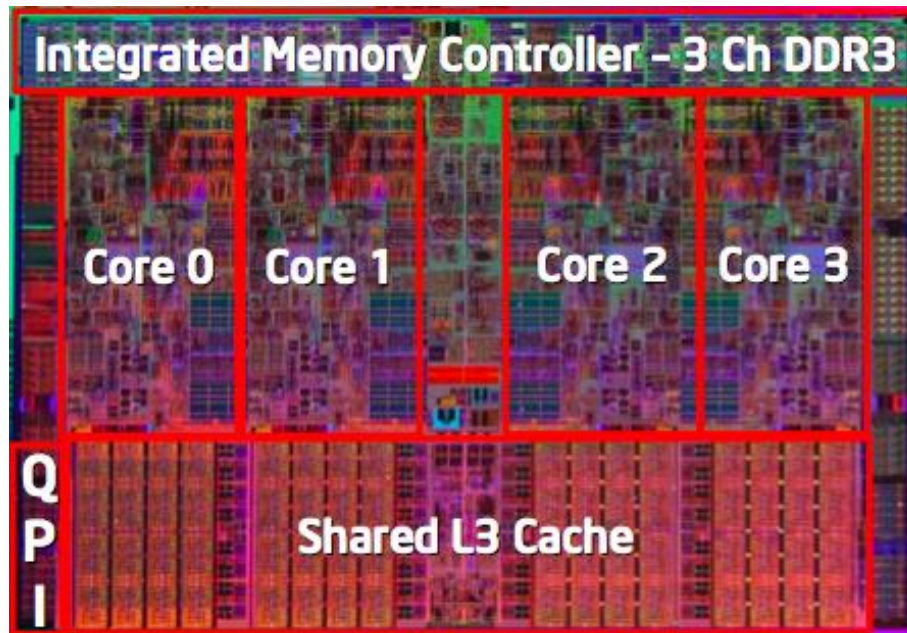
- Elimina disputa pela cache entre a unidade de busca/decodificação de instrução e a unidade de execução
 - Importante no pipeline de instruções



NÚMERO DE CACHES

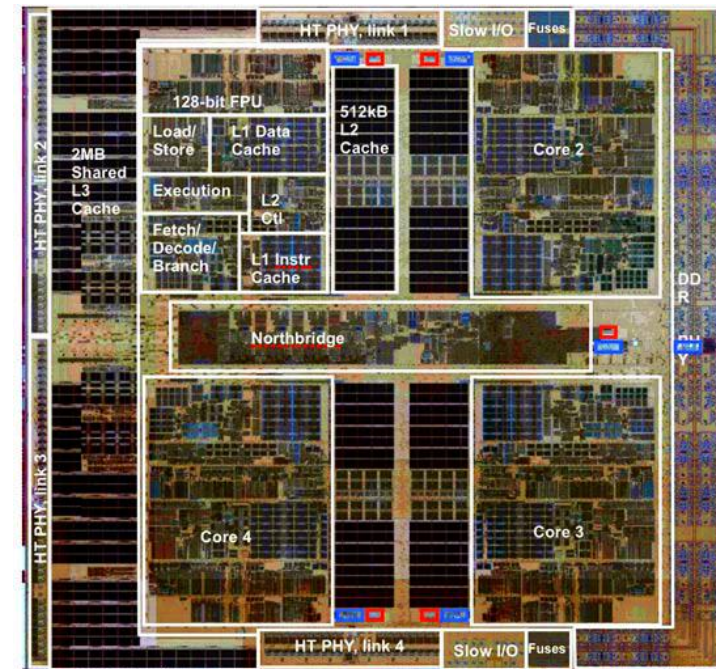
Intel Nehalem (Core i7)

- L1: 64KB (32KB+32KB por core)
- L2: 256KB (por core)
- L3: 8 MB (compartilhada)



AMD Opteron

- L1: 64 KB (por core)
- L2: 512 KB (por core)
- L3: 2 MB (compartilhada)



NÚMERO DE CACHES

Tempos de acesso aproximados do Core i7 Xeon 5500* (2009 – 45nm)

L1 CACHE hit,	~4 cycles (2.1 - 1.2 ns)
L2 CACHE hit,	~10 cycles (5.3 - 3.0 ns)
L3 CACHE hit, linha não compartilhada	~40 cycles (21.4 - 12.0 ns)
L3 CACHE hit, linha compartilhada com outro core	~65 cycles (34.8 - 19.5 ns)
L3 CACHE hit, modificada em outro core	~75 cycles (40.2 - 22.5 ns)
DRAM	~60 ns

* <https://www.intel.com/content/dam/develop/external/us/en/documents/performance-analysis-guide-181827.pdf>

CAPACIDADE DA CACHE

“Tamanho pequeno o suficiente para que o custo médio por bit fosse próximo do custo da memória principal isolada e grande o suficiente para que o tempo de acesso médio geral fosse próximo do tempo de acesso médio da cache isolada.”

Espaço

- A área disponível do chip e da placa limita o tamanho da cache
- Quanto maior a cache, maior é o número de portas envolvidas no endereçamento da cache

CAPACIDADE DA CACHE

Custo

- Mais cache é caro

Velocidade

- Mais cache é mais rápido (até certo ponto)
- Verificar dados na cache leva tempo

Impossível chegar a um único tamanho ideal

- L1 = 32KB (I/D)
- L2 = 256KB-512KB

Processador	Tipo	Ano de introdução	Cache L1 ^a	Cache L2	Cache L3
IBM 360/85	Mainframe	1968	16–32 kB	—	—
PDP-11/70	Minicomputador	1975	1 kB	—	—
VAX 11/780	Minicomputador	1978	16 kB	—	—
IBM 3033	Mainframe	1978	64 kB	—	—
IBM 3090	Mainframe	1985	128–256 kB	—	—
Intel 80486	PC	1989	8 kB	—	—
Pentium	PC	1993	8 kB/8 kB	256–512 kB	—
PowerPC 601	PC	1993	32 kB	—	—
PowerPC 620	PC	1996	32 kB/32 kB	—	—
PowerPC G4	PC/servidor	1999	32 kB/32 kB	256 kB a 1 MB	2 MB
IBM S/390 G6	Mainframe	1999	256 kB	8 MB	—
Pentium 4	PC/servidor	2000	8 kB/8 kB	256 kB	—
IBM SP	Servidor avançado/ supercomputador	2000	64 kB/32 kB	8 MB	—
CRAY MTA ^b	Supercomputador	2000	8 kB	2 MB	—
Itanium	PC/servidor	2001	16 kB/16 kB	96 kB	4 MB
Itanium 2	PC/servidor	2002	32 kB	256 kB	6 MB
IBM POWER5	Servidor avançado	2003	64 kB	1.9 MB	36 MB
CRAY XD-1	Supercomputador	2004	64 kB/64 kB	1 MB	—
IBM POWER6	PC/servidor	2007	64 kB/64 kB	4 MB	32 MB
IBM z10	Mainframe	2008	64 kB/128 kB	3 MB	24–48 MB
Intel Core i7 EE 990	Estação de trabalho/ servidor	2011	6 × 32 kB/ 32kB	1,5 MB	12 MB
IBM zEnterprise 196	Mainframe/ servidor	2011	24 × 64 kB/ 128 kB	24 × 1,5 MB	24 MB L3 192 MB L4

^a Dois valores separados por uma barra referem-se a caches de instrução e dados. ^b As duas caches são apenas de instrução; não há caches de dados.

TAMANHO DO BLOCO

Conforme o bloco cresce, aumenta a taxa de acerto por causa da vizinhança...

- ... mas aumentando muito diminuem os número de blocos (por causa da capacidade) e aumenta a substituição.

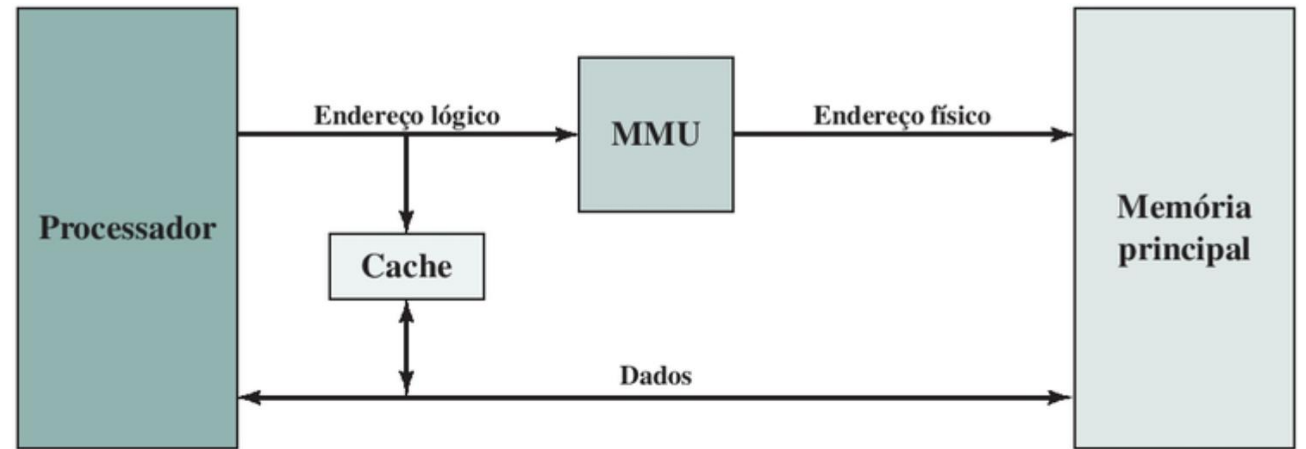
Escolher um bloco de maior tamanho, cada palavra adicional está mais distante da palavra usada e, portanto, tem menor probabilidade de ser utilizada em futuro próximo.

Arquiteturas x86 possuem blocos de 64 bytes. O processador POWER9 da IBM trabalha com blocos de 128 bytes.

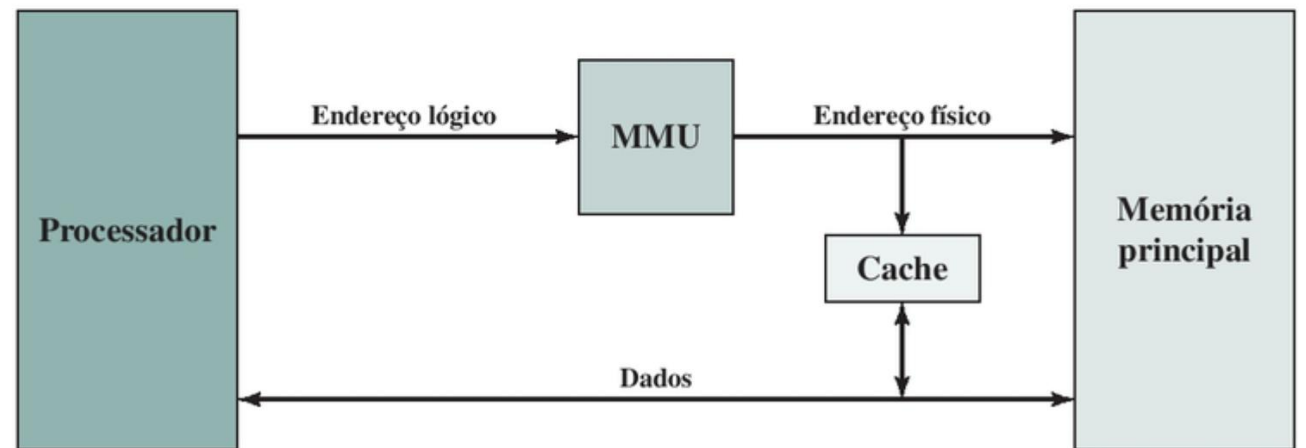
ENDEREÇO

Onde fica a cache?

- Entre o processador e a unidade de gerenciamento de memória virtual (MMU – *Memory Management Unit*)
- Entre MMU e a memória principal



(a) Cache lógica



(b) Cache física

ENDEREÇO

Cache lógica (cache virtual) armazena dados usando endereço virtual

- Processador acessa cache diretamente, não através da cache física
- Acesso à cache mais rápido, antes da tradução de endereço da MMU
- Endereços virtuais usam o mesmo espaço de endereços para diferentes aplicações
 - Deve esvaziar cache a cada troca de contexto

Cache física armazena dados usando endereços físicos da memória principal

TIPOS DE MAPEAMENTO

Como sabemos se um item de dado está na cache?

Se estiver, como encontrá-lo?

Para isso empregamos algumas técnicas de mapeamento:

- Mapeamento Direto
- Associativo por conjunto
- Totalmente associativo

MAPEAMENTO DIRETO

Cada bloco da memória principal é mapeado numa linha na cache

- Ou seja, se um bloco está na cache, ele deve estar em um local específico

Como a Cache é muito menor do que a memória principal, muitos endereços compartilham o mesmo bloco da memória cache

- Linha/bloco de 64 bytes
- Cache de 32 KB
 - 512 linhas
- Memória Principal de 8 GB
 - 128 M blocos
- Cada linha da cache vai ter 256K (262.144) blocos mapeados

MAPEAMENTO DIRETO

Parte do endereço procurado funciona como índice que informa qual linha da cache contém o dado procurado

Um endereço é dividido em

- Rótulo: identifica o bloco (t bits)
- **Linha**: identifica a posição (linha) do bloco na cache (b bits, $M = 2^b$)
 - 32 (2^5) linhas = 5 bits, 512 linhas = 9 bits, 16K linhas = 14 bits
- **Palavra**: identifica o deslocamento da palavra dentro do bloco (w bits, $K = 2^w$)
 - Bloco de 2 bytes = 1 bit, 64 bytes = 6 bits



MAPEAMENTO DIRETO

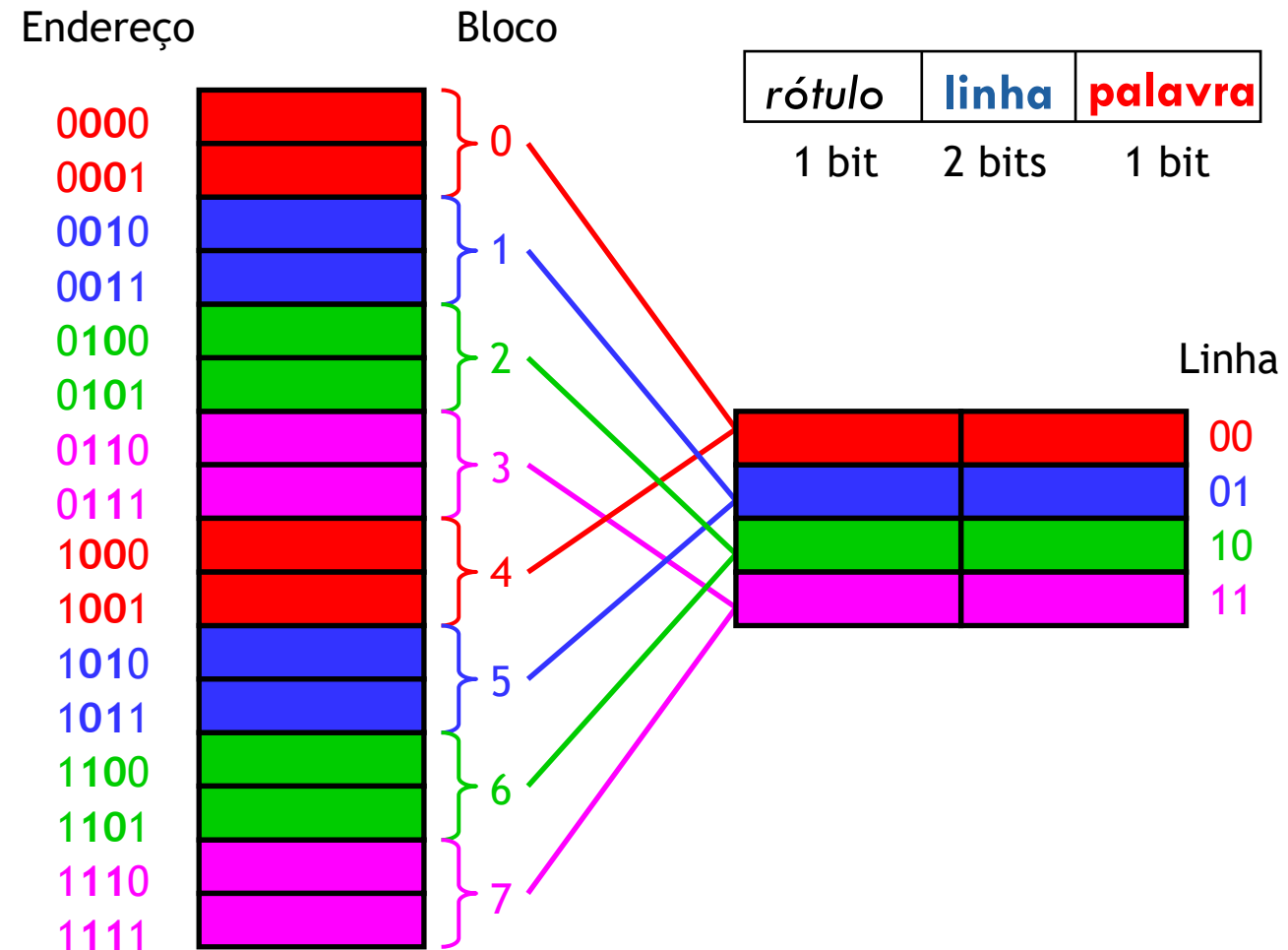
Bloco possui vários bytes
(localidade espacial)

A linha da cache é definida
pelos bits do endereço

rótulo	linha	palavra
--------	-------	---------

- Bloco 4 (endereços 8 e 9) é mapeado na linha 0 (00_2)
- Bloco 6 (endereços 12 e 13) é mapeado na linha 2 (10_2)

Rótulo é usado para identificar
que bloco está armazenado na
linha



MAPEAMENTO DIRETO

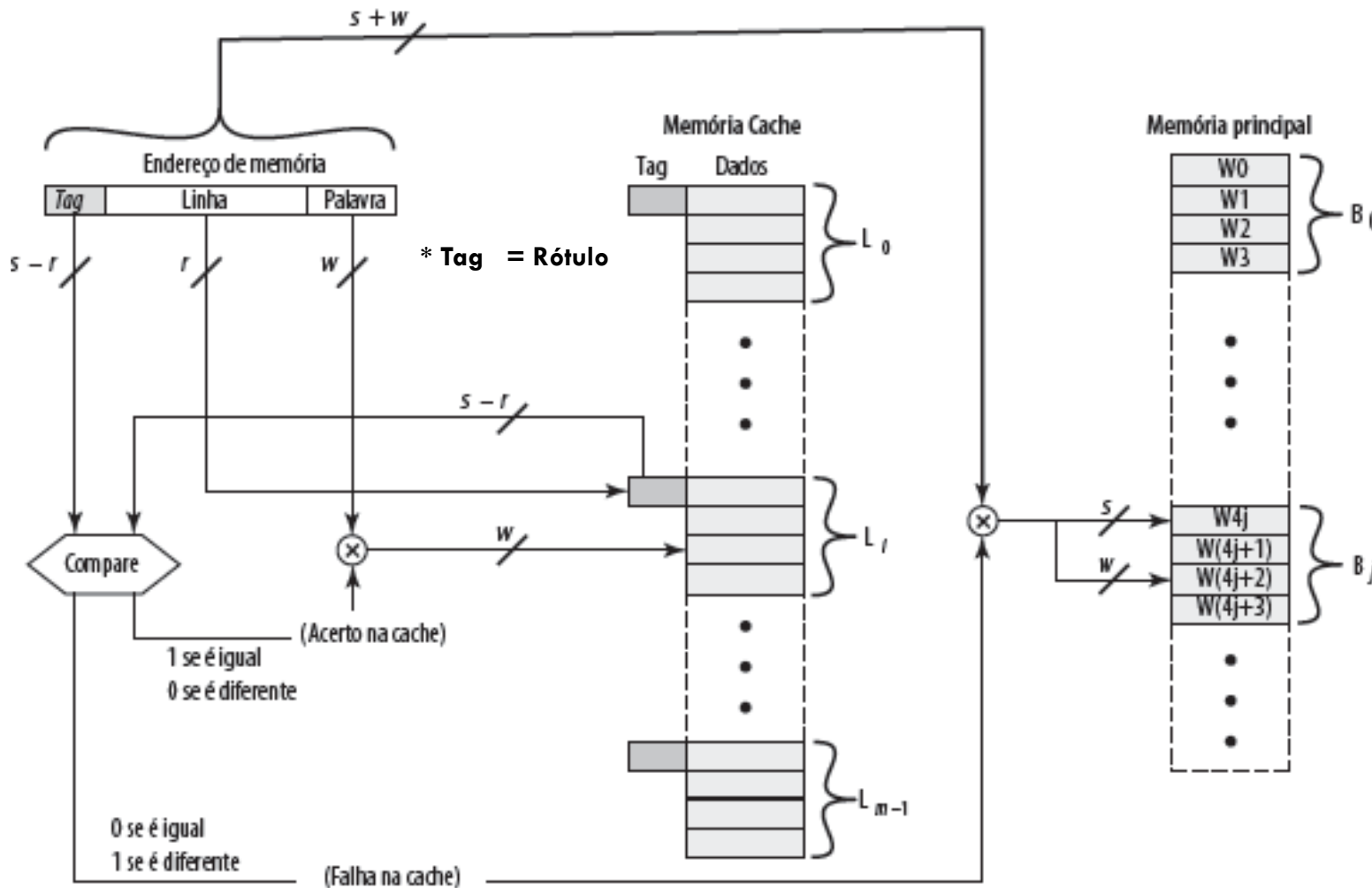
Mapeamento do bloco da memória principal na memória cache

$$i = j \% C$$

- i = número da linha da memória cache
- j = número do bloco da memória principal
- C = número de blocos na memória cache

Linha da cache	Blocos da memória principal
0	0, C, 2C, ..., $2^{t+b} - C$
1	1, C+1, 2C+1, ..., $2^{t+b} - C + 1$
...	...
C-1	C-1, 2C-1, 3C-1, ..., $2^{t+b} - 1$

MAPEAMENTO DIRETO



Tamanho de endereço = $(s + w)$ bits

Número de unidades endereçáveis = 2^{s+w} bytes

Tamanho de bloco = tamanho de linha = 2^w bytes

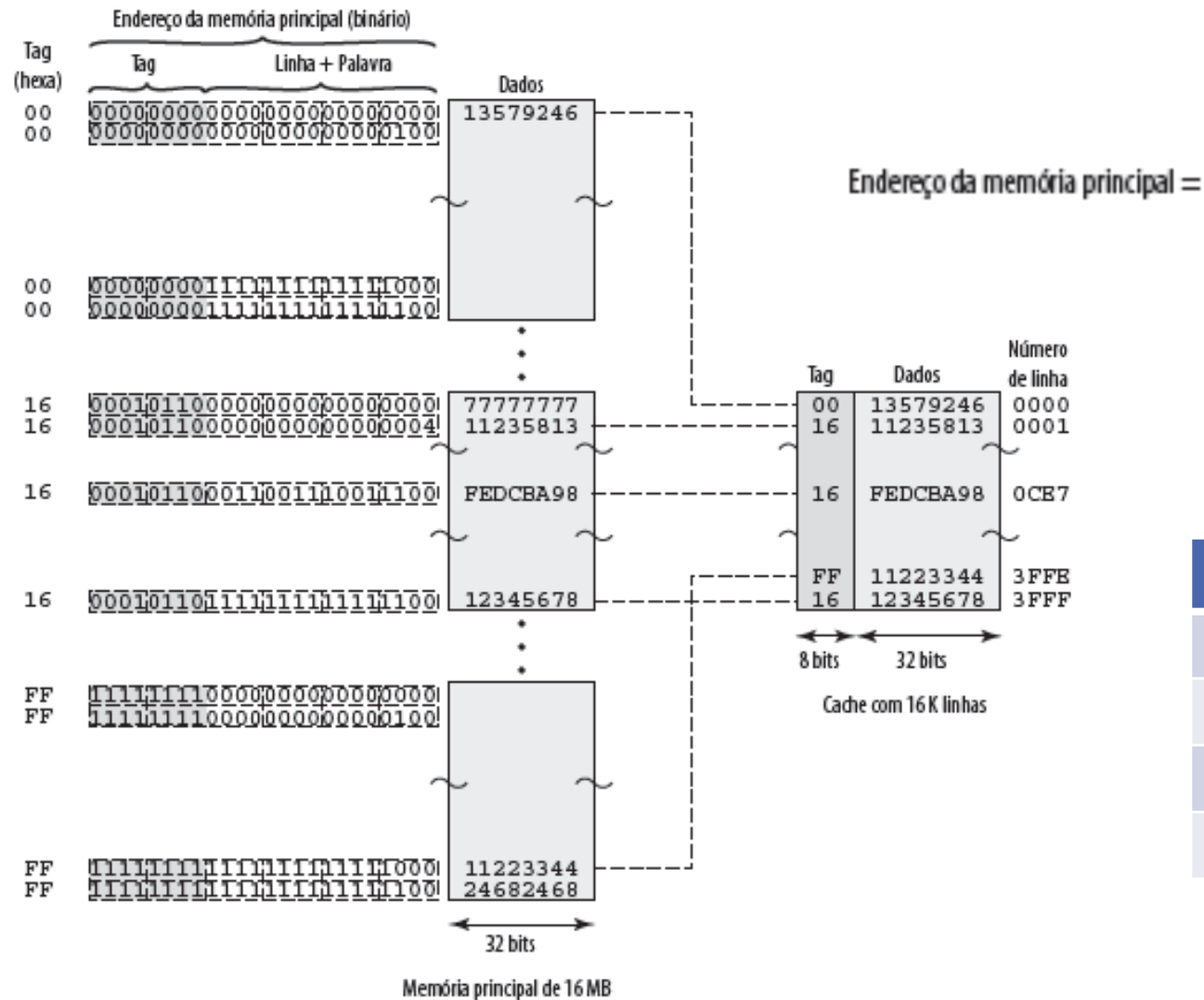
Número de blocos na MP = $2^{s+w} / 2^w = 2^s$

Número de linhas na cache = $m = 2^r$

Tamanho da Cache = 2^{r+w}

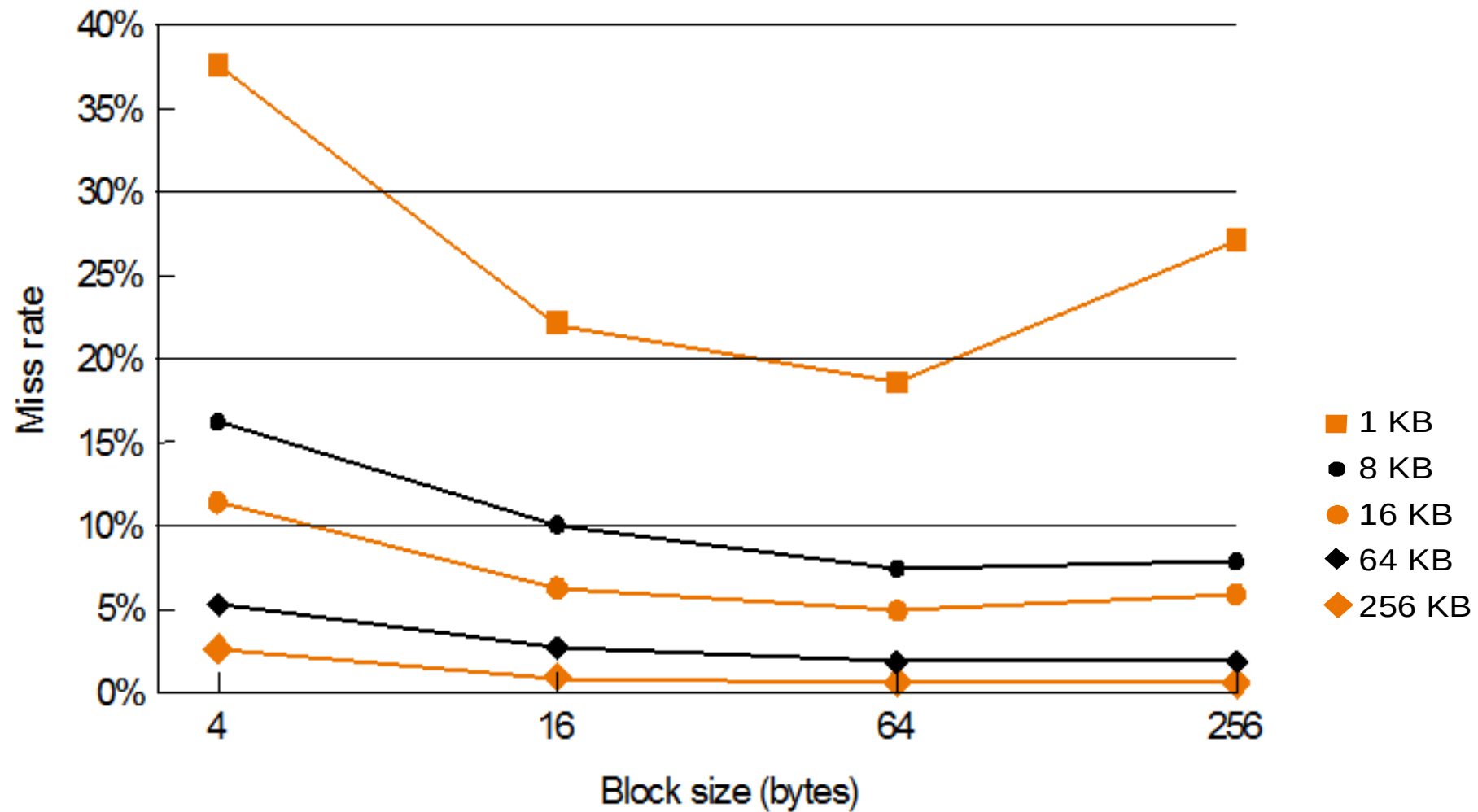
Tamanho do rótulo = $(s - r)$ bits

MAPEAMENTO DIRETO



Linha da cache	Endereço inicial do bloco de memória
0	000000, 010000, ..., FF0000
1	000004, 010004, ..., FF0004
...	...
C-1	00FFFC, 01FFFC, ..., FFFFFC

MAPEAMENTO DIRETO



MEMÓRIA CACHE: MAPEAMENTO DIRETO

Vantagens

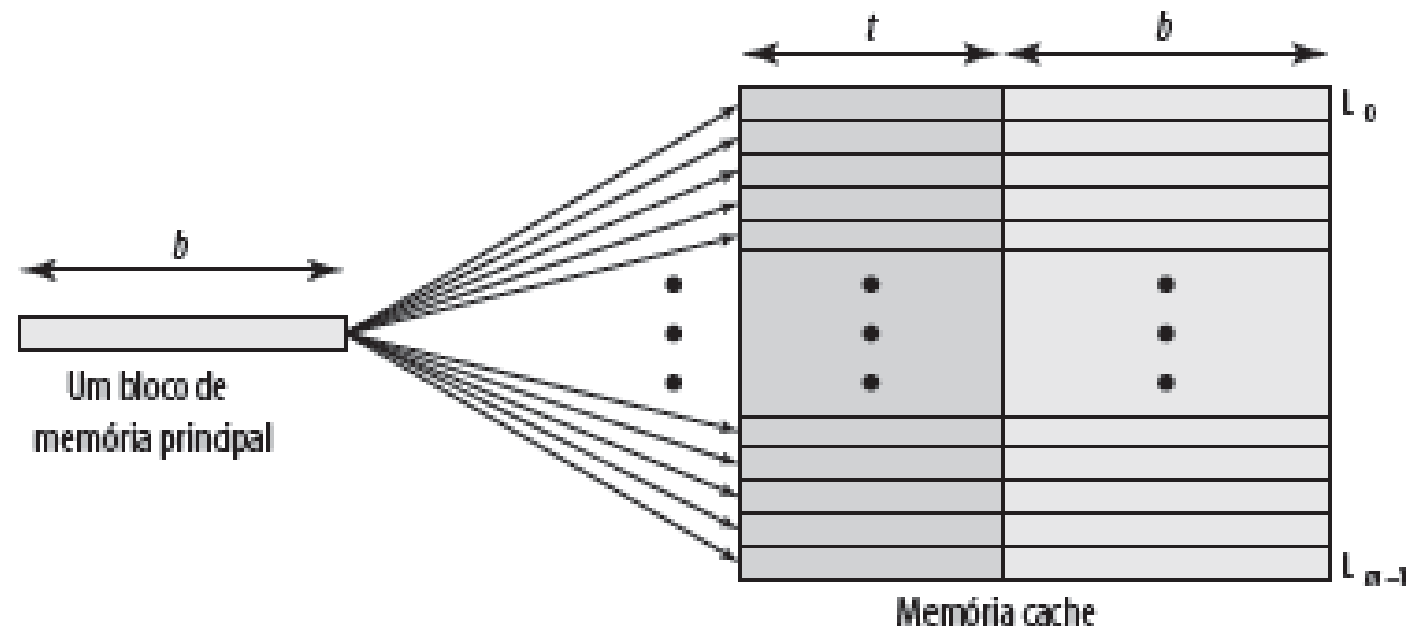
- Não há necessidade de algoritmo de substituição
- Hardware simples e de baixo custo
- Alta velocidade de operação

Desvantagens

- **Desempenho cai se acessos consecutivos são feitos a palavras de blocos diferentes mas que são mapeados na mesma linha da cache (falta de flexibilidade no mapeamento dos blocos)**
- É possível ter linhas na cache ociosas

MEMÓRIA CACHE: MAPEAMENTO TOTALMENTE ASSOCIATIVO

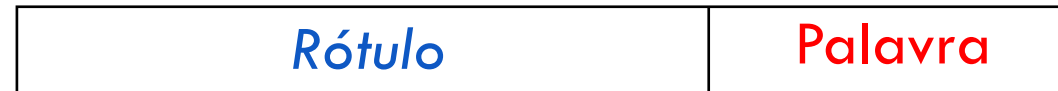
Cada linha da memória principal pode acessar qualquer linha da cache, pois cada linha da cache possui, associada a ela, o número da linha da memória principal que a ocupa



MEMÓRIA CACHE: MAPEAMENTO TOTALMENTE ASSOCIATIVO

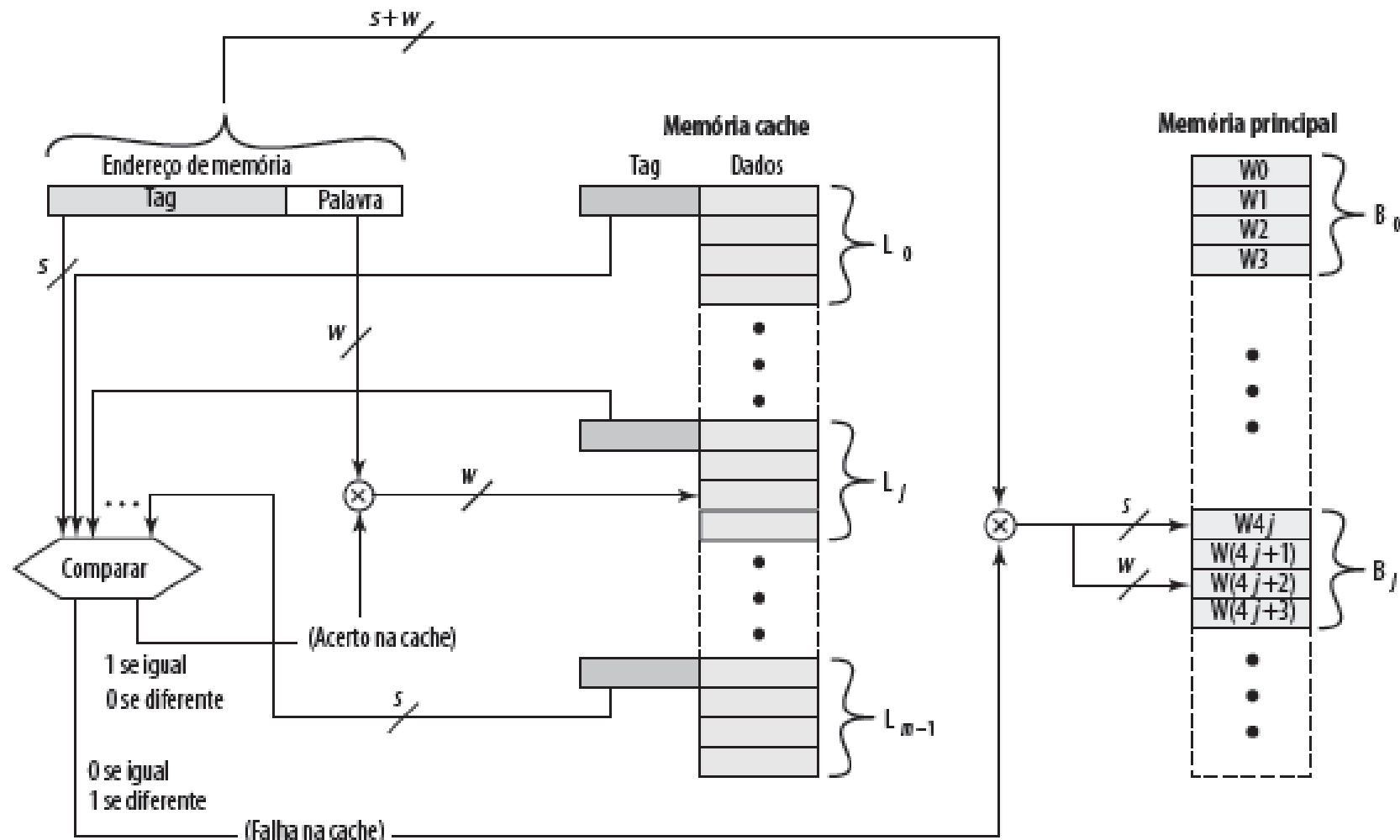
O rótulo tem que identificar um bloco da memória principal

- **Rótulo**: identifica o bloco (t bits)
- **Palavra**: identifica o deslocamento da palavra dentro do bloco (w bits, $K = 2^w$)

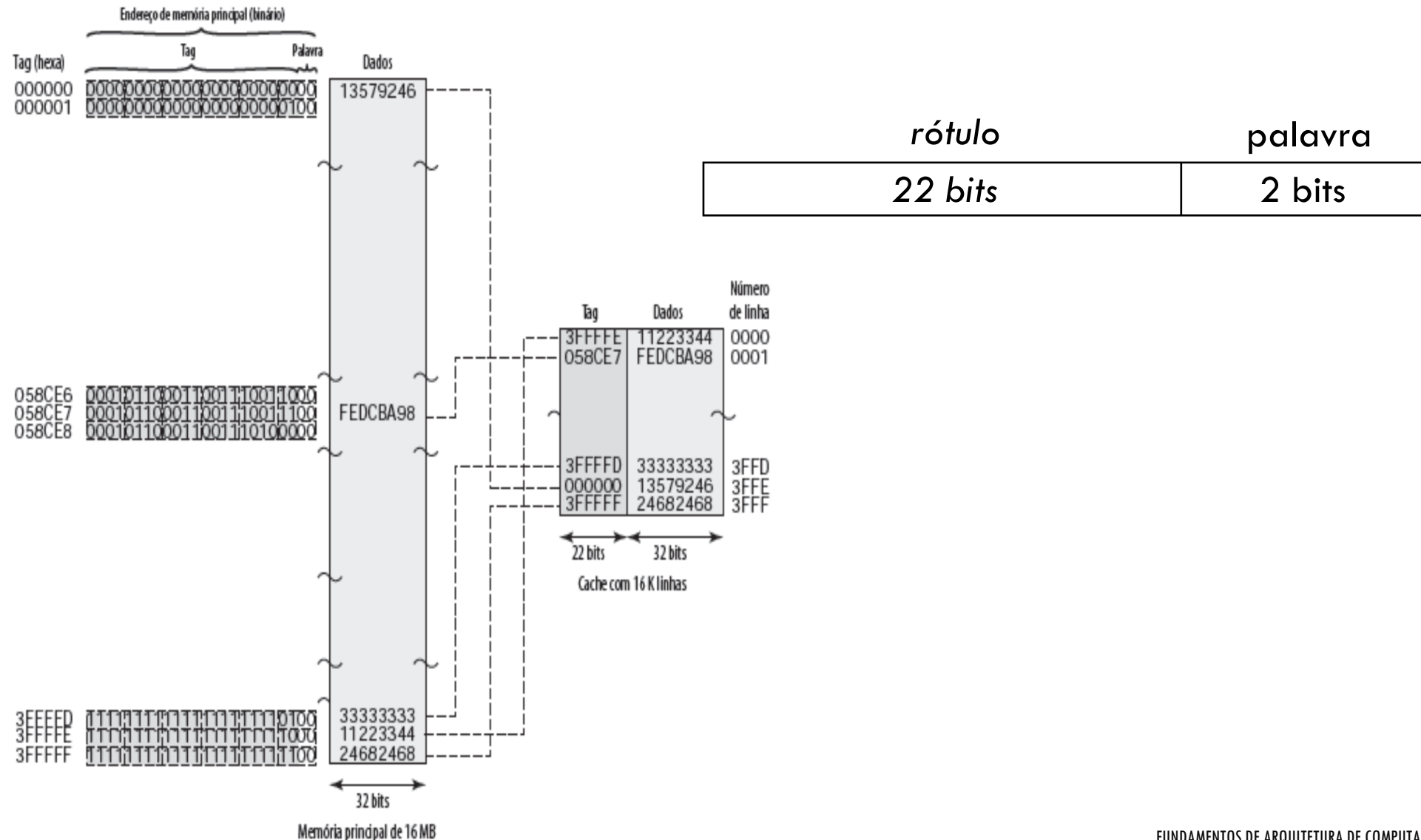


Cada bloco da cache deve ser examinado (comparado) quando for verificar se a cache contém o bloco do endereço desejado

MEMÓRIA CACHE: MAPEAMENTO TOTALMENTE ASSOCIATIVO



MEMÓRIA CACHE: MAPEAMENTO TOTALMENTE ASSOCIATIVO



MEMÓRIA CACHE: MAPEAMENTO TOTALMENTE ASSOCIATIVO

Vantagem

- Possui grande flexibilidade para a escolha da linha da cache

Desvantagem

- Necessita de algoritmo de substituição
- Complexidade do conjunto de circuitos necessários para comparação
- N comparações necessárias

MEMÓRIA CACHE: MAPEAMENTO ASSOCIATIVO POR CONJUNTO

Combina as vantagens do mapeamento direto e do mapeamento associativo e diminui as suas desvantagens

Cache é dividida em conjuntos (grupos), cada conjunto pode conter n blocos simultaneamente

Os n blocos são sempre os mesmos blocos pré-definidos da memória (similar ao mapeamento direto)

- Dentro do conjunto o bloco pode ocupar qualquer posição

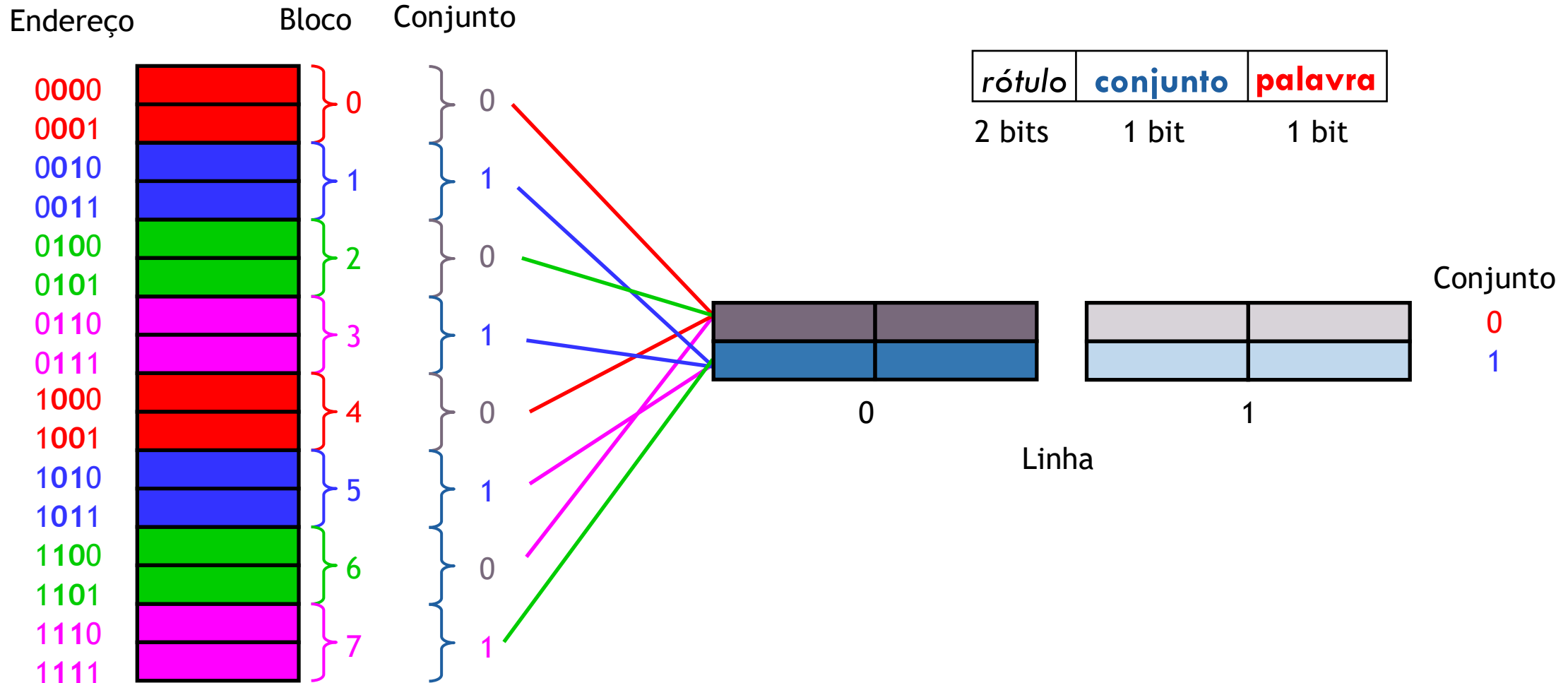
MEMÓRIA CACHE: MAPEAMENTO ASSOCIATIVO POR CONJUNTO

Um endereço é dividido em

- Rótulo: identifica o bloco (restante dos bits)
- **Conjunto**: identifica a posição do conjunto na cache (c bits, número de conjuntos = 2^c)
- **Palavra**: identifica o deslocamento da palavra dentro do bloco (p bits, tamanho do bloco = 2^p)



MEMÓRIA CACHE: MAPEAMENTO ASSOCIATIVO POR CONJUNTO



MEMÓRIA CACHE: MAPEAMENTO ASSOCIATIVO POR CONJUNTO

Cada bloco da memória principal é mapeado numa linha na cache

$$p = C/n$$
$$i = j \text{ módulo } p$$

i = número do conjunto da memória cache

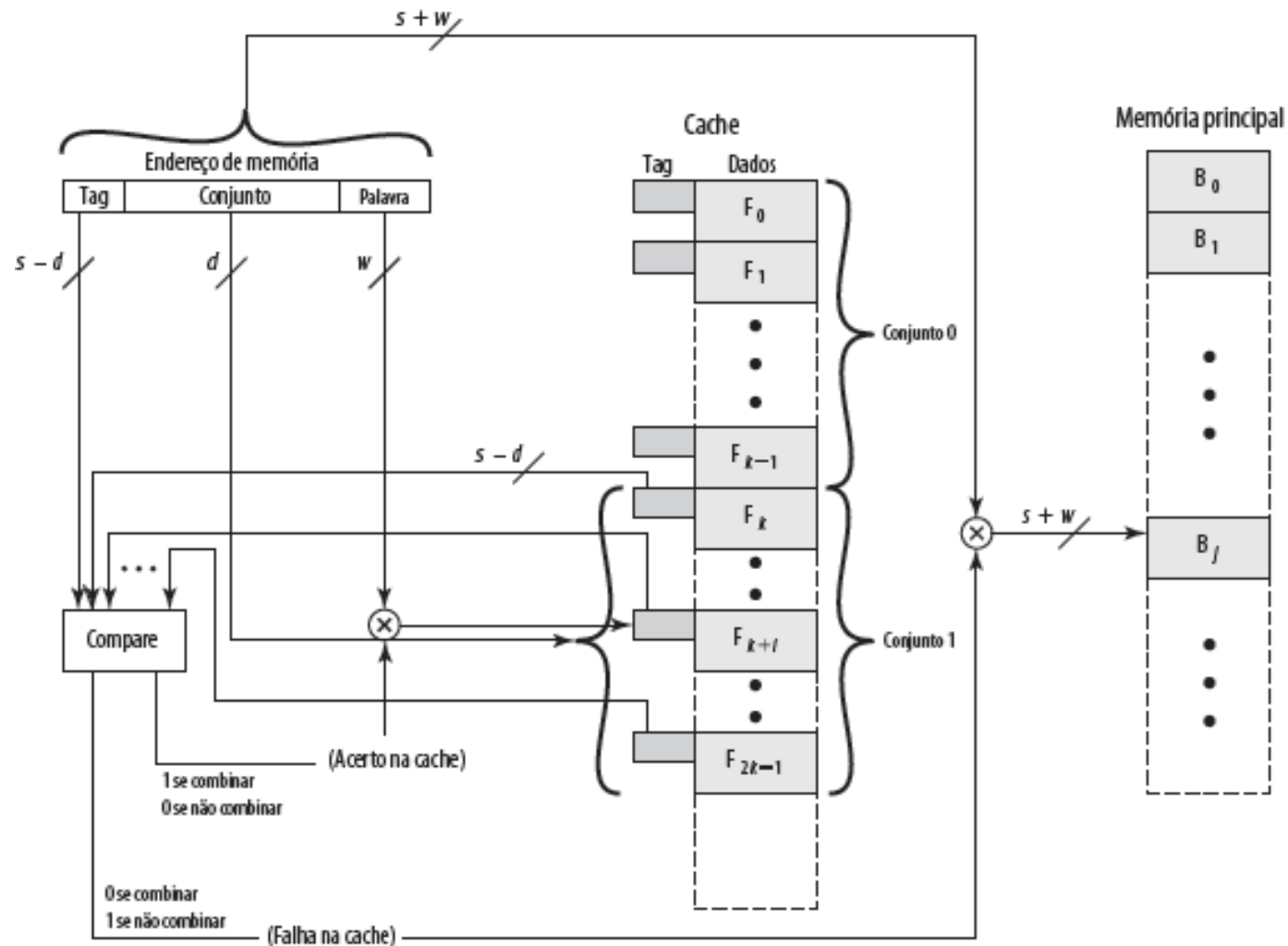
j = número do bloco da memória principal

C = número de blocos na memória cache

n = número de blocos de um conjunto

p = número de conjuntos da memória cache

MEMÓRIA CACHE: MAPEAMENTO ASSOCIATIVO POR CONJUNTO



Número de linhas no conjunto = k

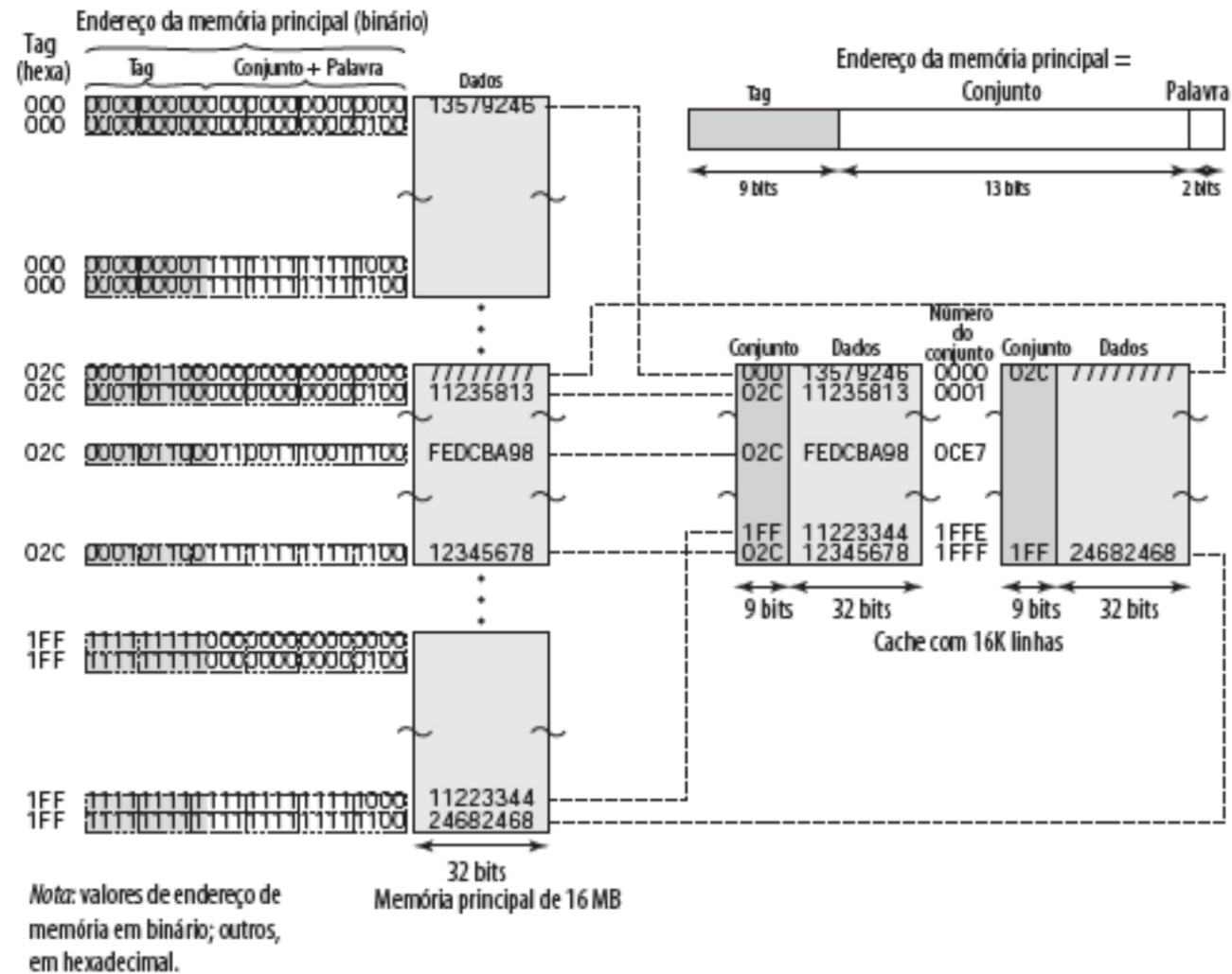
Número de conjuntos = $v = 2^d$

Número de linhas na cache = $k * v = k * 2^d$

Tamanho da cache = $k * 2^{d+w}$

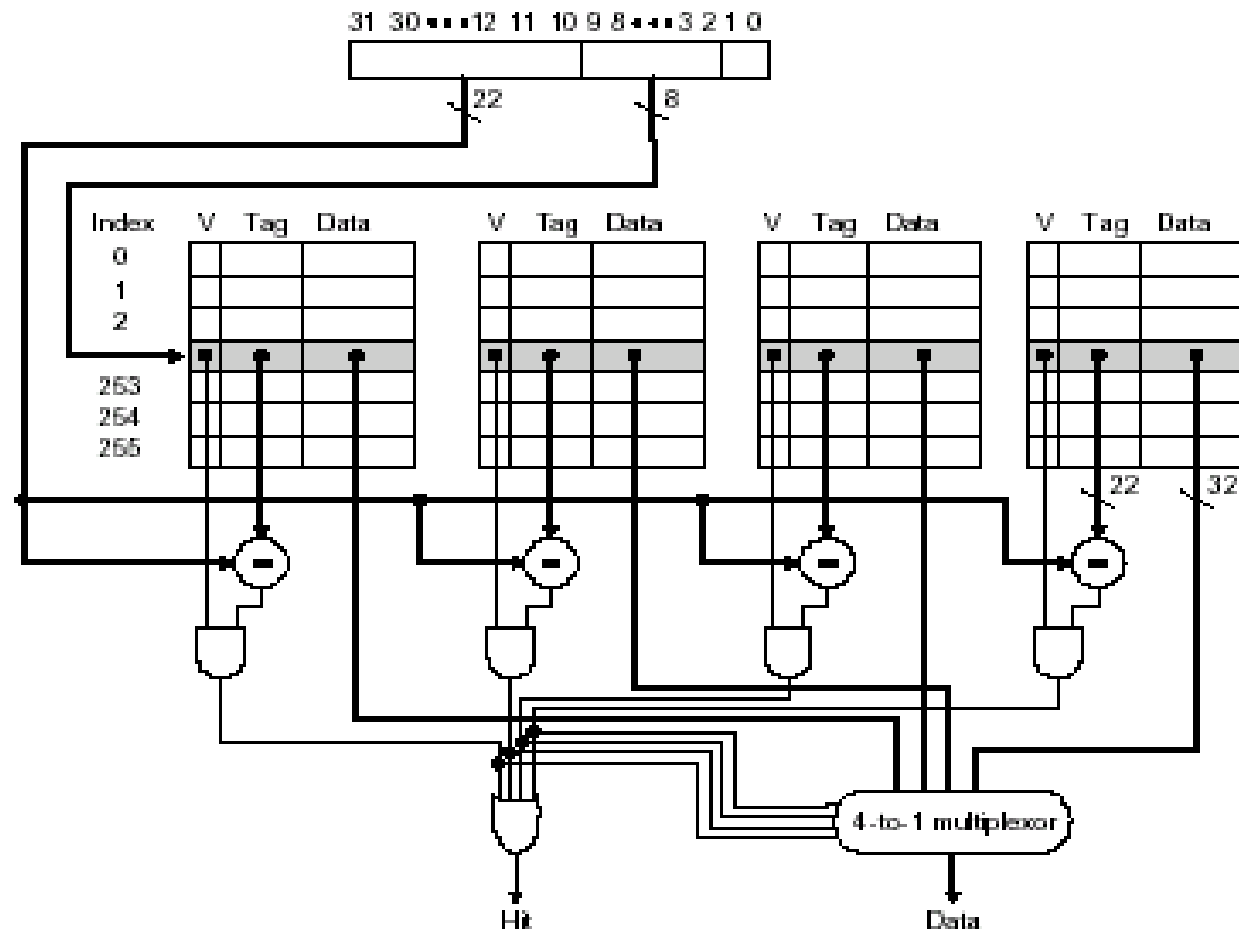
Tamanho da tag = $(s - d)$ bits

MEMÓRIA CACHE: MAPEAMENTO ASSOCIATIVO POR CONJUNTO

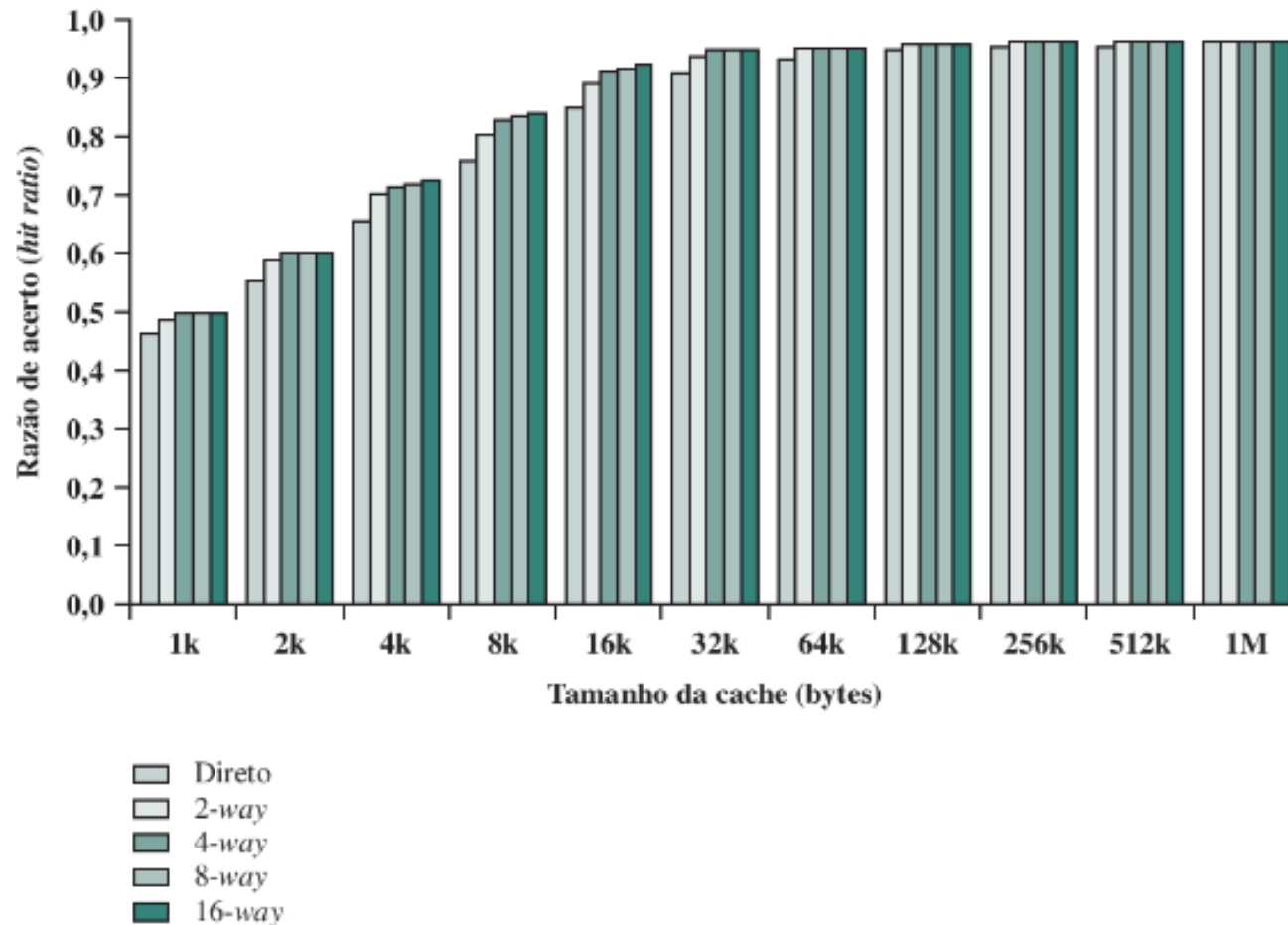


MEMÓRIA CACHE: MAPEAMENTO ASSOCIATIVO POR CONJUNTO

Cache Associativa por 4-vias (4-way)



MEMÓRIA CACHE: MAPEAMENTO ASSOCIATIVO POR CONJUNTO



Cache		
L1 Data	6 x 32 KBytes	8-way
L1 Inst.	6 x 64 KBytes	4-way
Level 2	6 x 512 KBytes	8-way
Level 3	2 x 8 MBytes	16-way

Cache		
L1 Data	6 x 32 KBytes	8-way
L1 Inst.	6 x 32 KBytes	8-way
Level 2	6 x 256 KBytes	4-way
Level 3	9 MBytes	12-way

MAPEAMENTO ASSOCIATIVO POR CONJUNTO

Vantagem

- Possui flexibilidade para a escolha da linha em comparação ao associativo
- Endereçamento direto utilizado para localizar o conjunto na cache

Desvantagem

- N comparadores são necessários
- Multiplexador atrasa os dados
- Necessidade de um algoritmo de substituição (herança da flexibilidade do totalmente associativo)

POLÍTICA DE ESCRITA

Problema: a memória principal pode ser utilizada tanto pela UCP quanto por dispositivos de E/S

- Alterações na cache invalidam informação na memória principal
- Alterações por dispositivos de E/S invalidam informação na cache
- Alterações em ambientes multi-processados com cache local invalidam as demais caches

Tipos

- Write-through
- Write-back

POLÍTICA DE ESCRITA WRITE-THROUGH

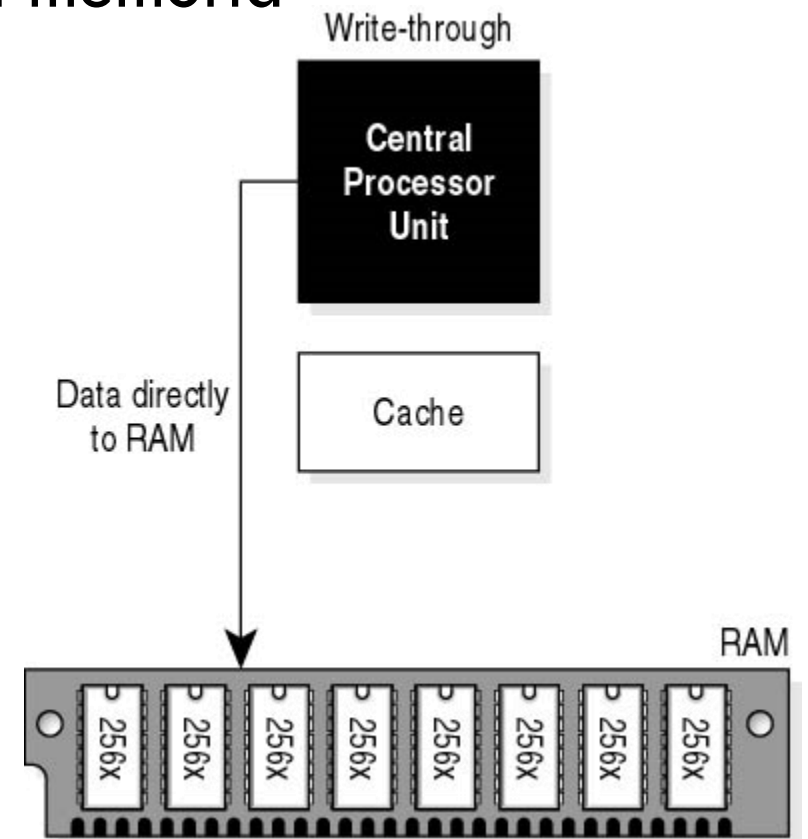
Cada escrita na cache gera uma escrita na memória principal

Múltiplas CPUs podem monitorar o tráfego da memória principal para manter a cache local (à CPU) atualizada

- Muito tráfego
- Atrasa as escritas

Somente 15% das referências à memória são de escrita

- Em computação de alto desempenho este valor pode chegar a 33% (multiplicação de vetores) e a 50% (transposição de matrizes)



POLÍTICA DE ESCRITA WRITE-THROUGH

Vantagens

- Fácil de implementar
- Um miss nunca resulta em escritas na memória
- A memória tem sempre a informação atualizada

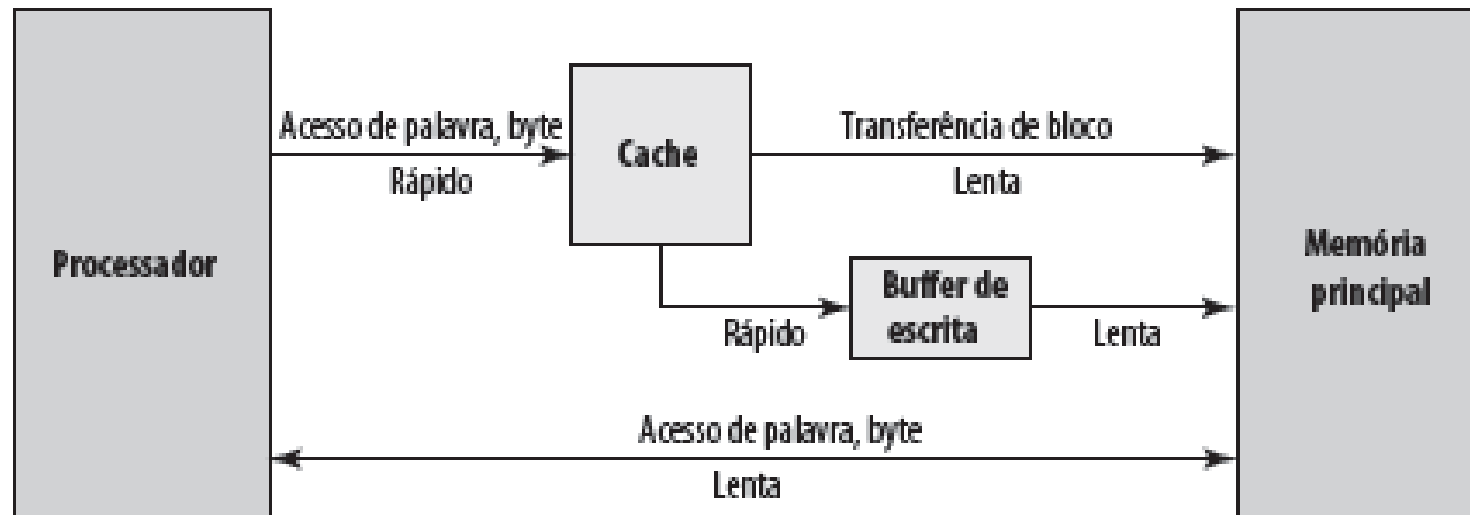
Desvantagens

- A escrita é lenta (mesma velocidade da MP)
- Cada escrita necessita de um acesso à memória (= mais largura de banda da memória)

POLÍTICA DE ESCRITA WRITE-THROUGH

Write-through com buffer

- Escrita na cache resulta em dado armazenado em buffer até ser escrito (em paralelo) pela controladora na MP
- Processadores ARM



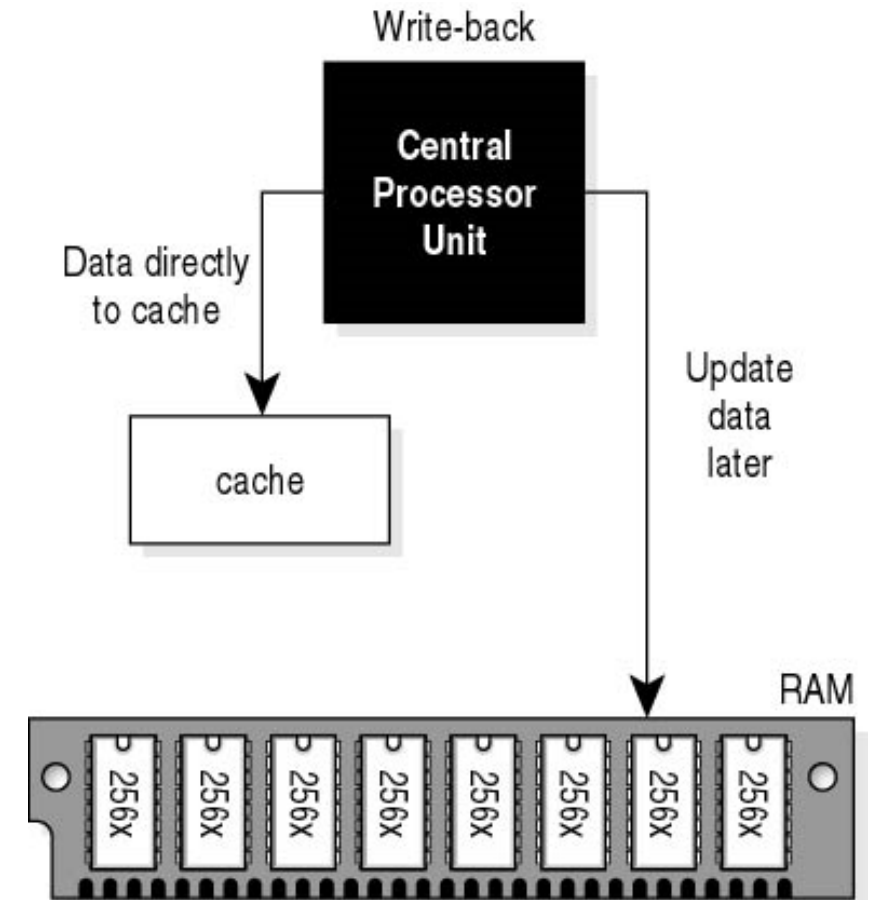
POLÍTICA DE ESCRITA WRITE-BACK

Atualizações feitas inicialmente apenas na cache

Bit de alteração (*dirty bit*) é setado em 1 quando bloco alterado

Se o bloco deve ser substituído, escreve na memória principal apenas se o bit alteração estiver marcado

E/S deve acessar a memória principal através da cache



POLÍTICA DE ESCRITA WRITE-BACK

Vantagens

- A escrita ocorre à velocidade da cache
- Escritas múltiplas de um endereço → apenas uma escrita na memória
- Consome menos largura de banda

Desvantagens

- Difícil de implementar
- Nem sempre existe consistência entre os dados existentes na cache e na memória
- Leituras de blocos de endereços na cache podem resultar em escritas de blocos de endereços "dirty" na memória

POLÍTICA DE ESCRITA NA FALHA AO ESCREVER

O que acontece durante a gravação de um dado na cache quando o seu bloco não está na cache?

- *Write allocate*
 - O bloco é trazido da memória principal para a cache o dado é atualizado na cache
 - Tipicamente utilizado no método *write-back*, pois espera-se que a próxima escrita (ou até mesmo leitura) seja no mesmo bloco
- *Non-write allocate*
 - O bloco é modificado na memória principal e a cache não é alterada
 - Tipicamente utilizado no método *write-through*, pois não há vantagem na próxima escrita já que a escrita é sempre realizada também na memória principal

ALGORITMO DE SUBSTITUIÇÃO

Mapeamento direto

- Sem escolha
- Cada bloco mapeado apenas a uma linha
- Substitui essa linha

Caches Associativas

- Algoritmo implementado no hardware (velocidade)
- LRU (Least Recently Used)
 - Bloco menos recentemente usado (bit de uso)
- FIFO (First-In-First-Out)
 - Substitui bloco que está na cache há mais tempo
- LFU (Least Frequently Used)
 - Substitui bloco que teve menos acertos
- Aleatório

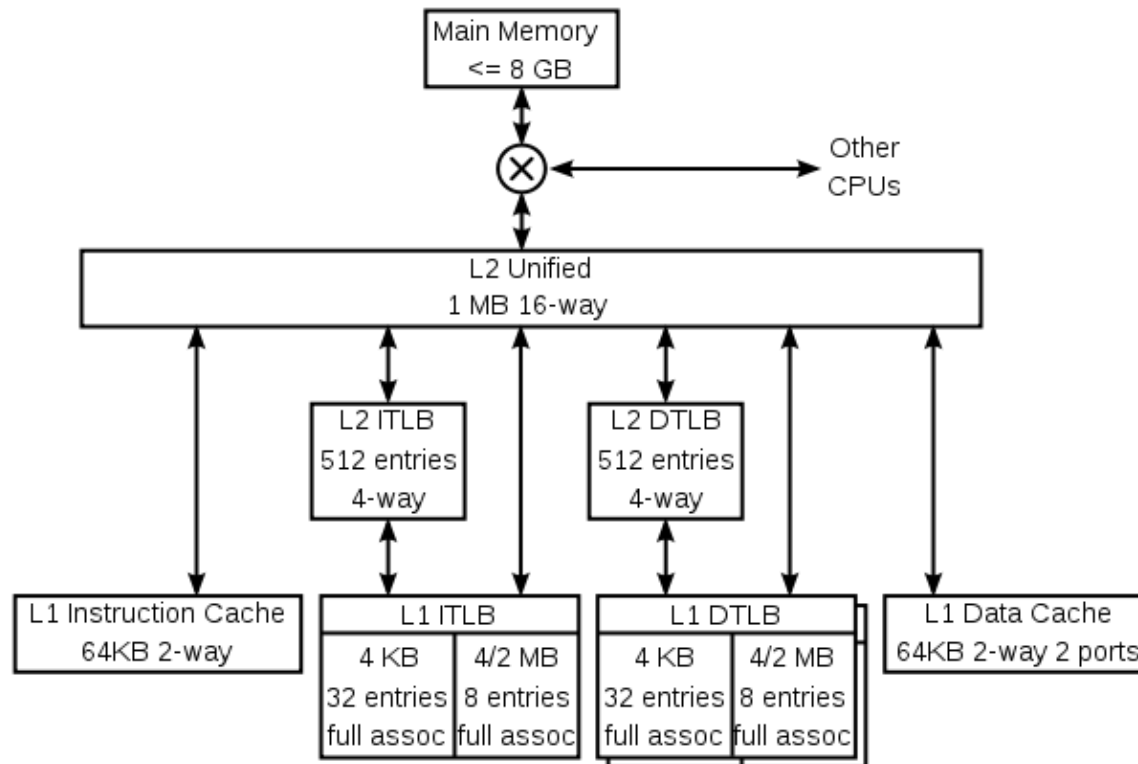
ALGORITMO DE SUBSTITUIÇÃO

Associatividade	2-way		4-way		8-way	
Tamanho	LRU	Aleatório	LRU	Aleatório	LRU	Aleatório
16KB	5.2%	5.7%	4.7%	5.3%	4.4%	5.0%
64KB	1.9%	2.0%	1.5%	1.7%	1.4%	1.5%
256KB	1.15%	1.17%	1.13%	1.13%	1.12%	1.12%

Estudos mostram que o Aleatório é o método que apresenta um desempenho levemente inferior ao do LRU

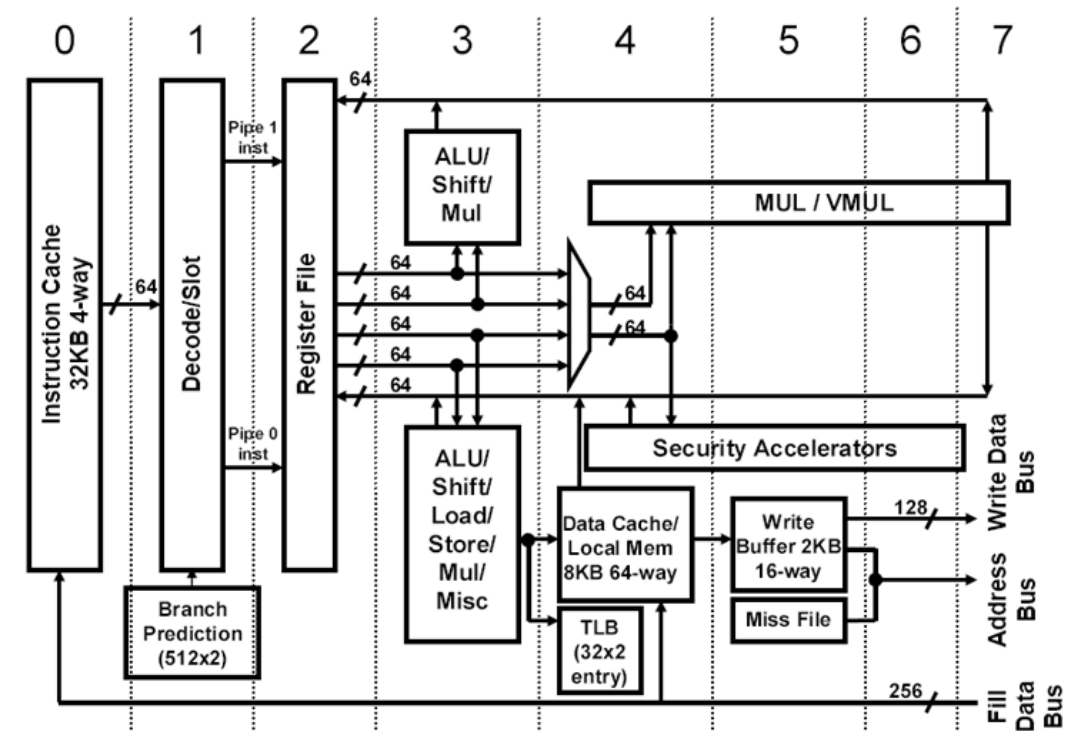
EXEMPLOS DE MEMÓRIA CACHE

K8 core no CPU AMD Athlon 64



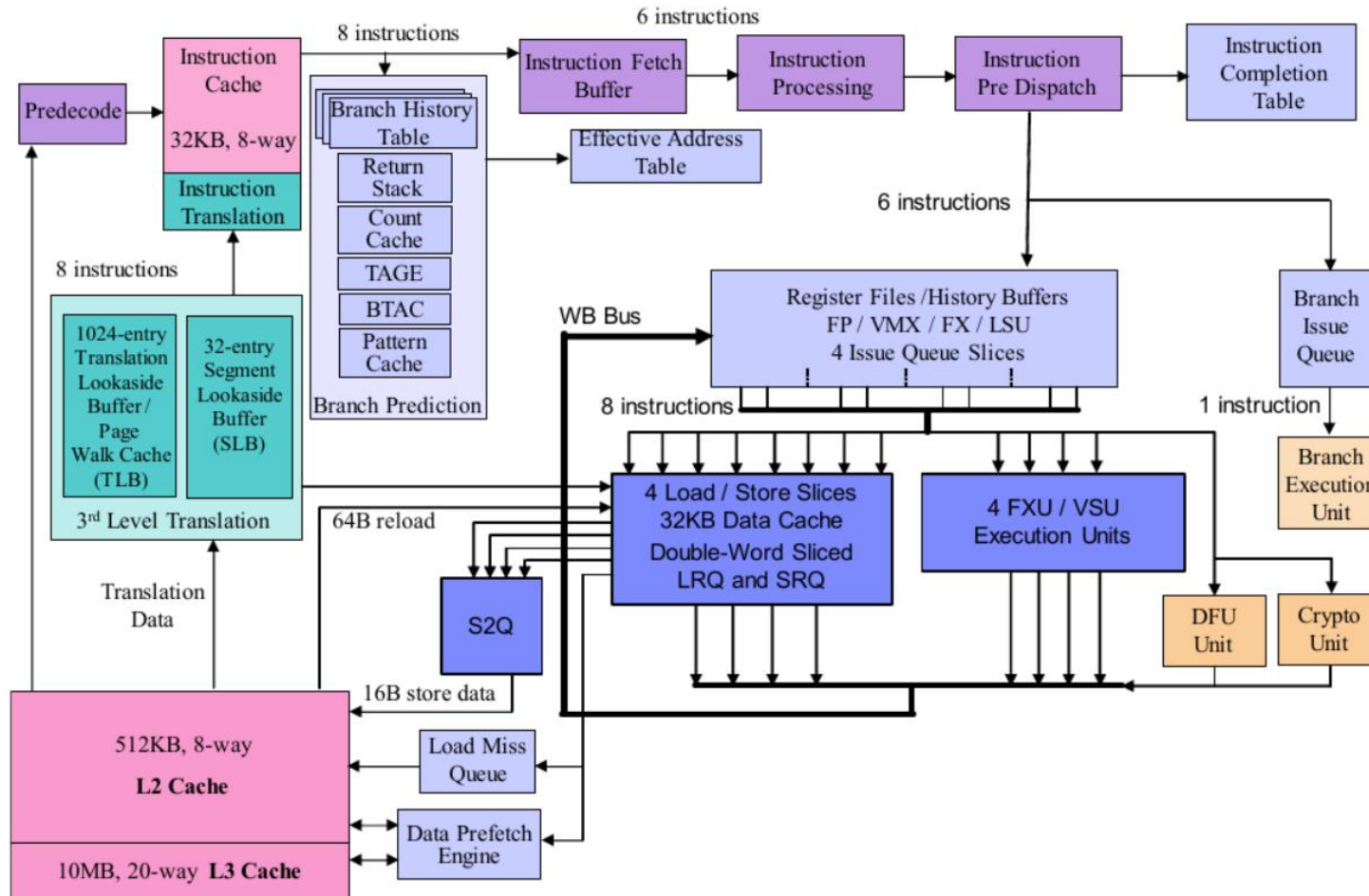
cnMIPS64 CPU Pipeline

32KB 4-way associative L1 Instruction cache

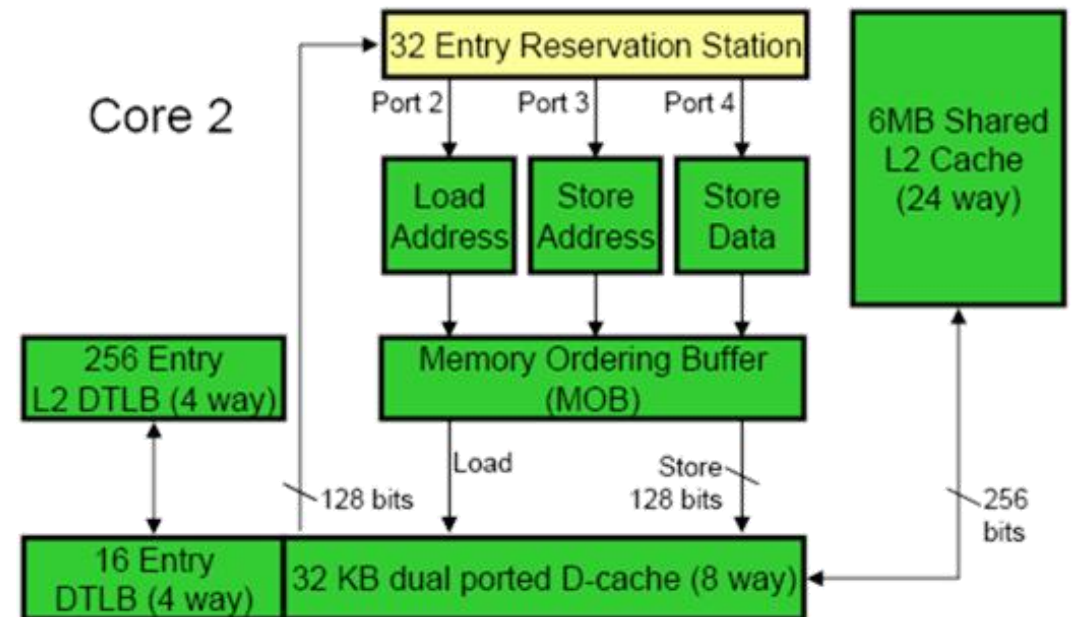
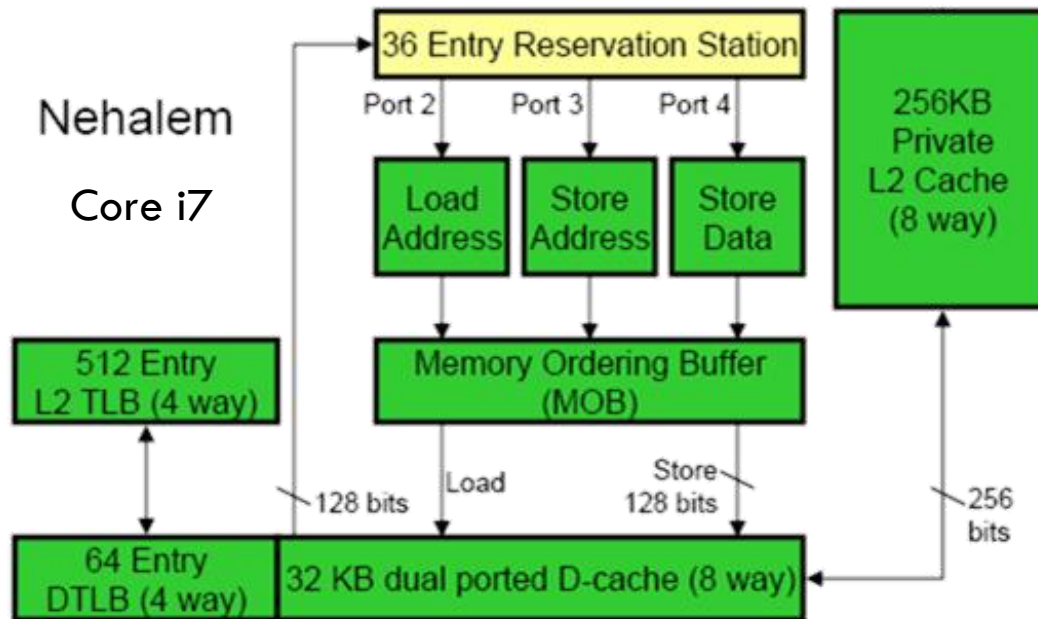


EXEMPLOS DE MEMÓRIA CACHE

POWER9 IBM



EXEMPLOS DE MEMÓRIA CACHE



EXEMPLOS DE MEMÓRIA CACHE

Núcleo	Tipo de cache	Tamanho de cache (kB)	Tamanho da linha de cache (palavras)	Associatividade (linhas por conjunto)	Local	Tamanho do buffer de escrita (palavras)
ARM720T	Unificada	8	4	4 linhas por conjunto	Lógico	8
ARM920T	Separada	16/16 D/I	8	64 linhas por conjunto	Lógico	16
ARM926EJ-S	Separada	4-128/4-128 D/I	8	4 linhas por conjunto	Lógico	16
ARM1022E	Separada	16/16 D/I	8	64 linhas por conjunto	Lógico	16
ARM1026EJ-S	Separada	4-128/4-128 D/I	8	4 linhas por conjunto	Lógico	8
Intel StrongARM	Separada	16/16 D/I	4	32 linhas por conjunto	Lógico	32
Intel Xscale	Separada	32/32 D/I	8	32 linhas por conjunto	Lógico	32
ARM11 36-JF-S	Separada	4-64/4-64 D/I	8	4 linhas por conjunto	Físico	32